Density Decomposition of Multilayer Graphs

Jiaqi Jiang, Rong-Hua Li, Yalong Zhang

Beijing Institute of Technology, Beijing, China;
jiaqijiang@bit.edu.cn; lironghuabit@126.com; yalong-zhang@qq.com

***Abstract*—Multilayer graphs have emerged as a powerful model for representing complex systems with diverse types of interactions. Identifying cohesive subgraphs in such graphs is a fundamental task with broad applications in community detection, fraud analysis, and e-commerce recommendation. However, existing models either lack explicit density guarantees or fail to preserve across-layer structural cohesiveness. To address these limitations, in this paper, we propose a novel $\mathbf{k}$-dense subgraph model, denoted as $D_{\mathbf{k}}$, which enforces layer-wise density constraints. Given a density vector $\mathbf{k} = [k_\ell]_{\ell \in L} \in \mathbb{Z}^{|L|}$, our model guarantees that the induced subgraph on each layer satisfies a minimum density threshold k_ℓ.**

We further study the density decomposition problem: computing all non-empty $D_{\mathbf{k}}$s for feasible density vectors $\mathbf{k}$. To this end, we design a suite of efficient algorithms. For computing a single $D_{\mathbf{k}}$, we first propose SLFBE, which uses network flow techniques to extract layer-wise subgraphs satisfying density thresholds. We then develop LCVSP, which combines $\mathbf{k}$-core pruning and layer-constrained vertex set propagation to eliminate irrelevant vertices and accelerate convergence. For full density decomposition, we leverage the *density lattice* structure and propose HPDD, which incrementally constructs subgraphs by extending partial vectors layer by layer, enabling localized exploration and early pruning. To further improve scalability, we develop the HPDD+ algorithm, which adopts a recursive depth-first hierarchical strategy to lower memory cost. Additionally, HPDD+ employs an adaptive upper-bounded divide-and-conquer approach that eliminates redundant computations. Extensive experiments on 9 real-world multilayer graphs demonstrate that our model discovers significantly higher-quality subgraphs than existing methods, while our algorithms exhibit superior efficiency and scalability.

I. Introduction

Multilayer graphs have emerged as a powerful abstraction for modeling complex interconnected systems where entities interact through multiple types of relationships. Examples span diverse domains, including social networks [1], [2], biological systems [3], [4], and financial transaction graphs [5], where each layer captures a different type of interaction or context. Therefore, mining cohesive subgraphs in these structures is a fundamental problem in acdemic research. In these settings, discovering cohesive subgraphs, defined as groups of entities that are densely connected across multiple layers, is a fundamental problem with broad applications in community detection [6], fraud analysis [7], [8], and urban transportation analysis [9].

To tackle this important task, several multilayer cohesive subgraph models have been proposed. The multilayer $\mathbf{k}$-core with $\mathbf{k} = [k_1, k_2, \cdots, k_{|L|}]$ [10] enforces layer-wise degree thresholds, requiring that each node has at least k_ℓ neighbors in each layer ℓ. The d-CC model [11] generalizes this by enforcing a uniform degree threshold d over a specified subset of layers. FirmCore [12] introduces flexibility by requiring each node to satisfy the k-degree constraint in at least λ arbitrary layers. FocusCore [13] extends this further, combining strict layer constraints on a focus set $\mathcal{F}$ with loose support across λ layers. FirmTruss [14] imposes truss-based constraints, requiring adjacent vertices to participate in at least $k-2$ common triangles across multiple layers.

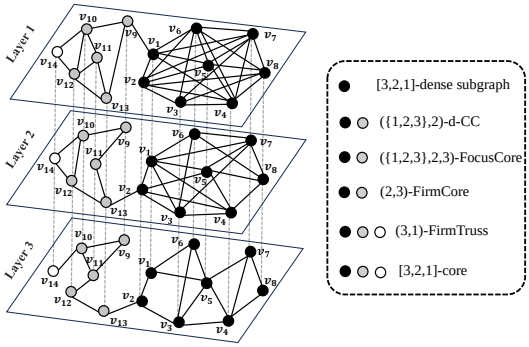


Fig. 1. A toy multilayer graph with 3 layers and illustration of existing models

However, these models often exhibit two critical limitations: (1) **Lack of layer-wise density guarantees.** Existing models rely primarily on degree-based (triangle-based) constraints, which may not guarantee layer-wise minimum density in a strict sense. As a result, even if the output subgraph appears structurally cohesive, it may still lack sufficient edge density. (2) **Lack of cross-layer structural cohesiveness.** Most models either apply uniform degree thresholds (*i.e.*, k) across layers or relax constraints to only a subset of layers. As a result, the extracted subgraphs tend to be cohesive in a few layers but loose or disconnected in others, lacking balanced structural consistency across the multilayer graph.

To address these limitations, we propose a novel $\mathbf{k}$-dense subgraph model for multilayer graphs, denoted as $D_{\mathbf{k}}$, which enforces layer-wise density lower bound $\rho(G_\ell[S]) = \frac{E_\ell[S]}{V_\ell[S]} > k_\ell$ for each layer ℓ. Compared to prior models, $D_{\mathbf{k}}$ more accurately captures dense communities across all layers. For example, as illustrated in Fig. 1, the community extracted by $D_{\mathbf{k}}$ that is densely connected in every layer, which achieves density of 3.12 (Layer 1), 2.37 (Layer 2), and 1.62 (Layer 3). In comparison, existing models produce communities that include sparse regions, leading to significantly lower density. For example, the communities discovered by d-CC, FocusCore, and FirmCore yield density of 2.53, 1.92, 1.46 across three layers, respectively, while $\mathbf{k}$-core and FirmTruss produce even sparser results with density of 2.50, 1.92, 1.42.

The ability of $D_{\mathbf{k}}$ to identify multilayer densest subgraphs enables impactful real-world applications where cohesiveness must be maintained simultaneously across diverse interaction

types. In biological networks, as validated in our experiments (Exp-9), $D_{\mathbf{k}}$ uncovers dynamic functional systems by capturing protein modules that are densely coupled across physical, genetic, and co-expression layers. Unlike degree-based models that often prune biologically integral but topologically peripheral nodes (e.g., transport machineries), $D_{\mathbf{k}}$ preserves these structures through global density guarantees. Similarly, in financial surveillance, our case study (Exp-10) demonstrates that $D_{\mathbf{k}}$ is critical for detecting organized fraud rings that exhibit consistently high density across multiple layers. While traditional methods fail to handle the heterogeneity of such criminal patterns, often missing valid fraud nodes due to strict local degree thresholds, $D_{\mathbf{k}}$ successfully isolates these hidden and cross-layer coordinated groups.

While the $D_{\mathbf{k}}$ provides a rigorous way to capture multilayer cohesion, each $D_{\mathbf{k}}$ represents only a single density configuration. In real multilayer systems, cohesive patterns often vary across layers and density scales. To obtain a holistic view of how these patterns evolve and interact, we aim to decompose the multilayer graph into all non-empty $D_{\mathbf{k}}$s, leading to the **density decomposition of multilayer graphs** problem. Given a graph G with L layers, our goal is to compute all non-empty $D_{\mathbf{k}}$s for all feasible density vectors $\mathbf{k}$. This problem poses two unique challenges that distinguish it from traditional single-layer density decomposition [15], [16].

The first challenge lies in the **intrinsic layer coupling**, which complicates the identification of a single $D_{\mathbf{k}}$. Unlike single-layer settings where density is optimized locally, dense regions in multilayer graphs are often spatially disjoint (structural mismatch). Consequently, enforcing a unified vertex set for $D_{\mathbf{k}}$ creates a global dependency: vertex removals in one layer can trigger density violations in others, leading to cascading updates. Crucially, distinct from multilayer $\mathbf{k}$-core mining which resolves such cascades via efficient local degree checks, $D_{\mathbf{k}}$ requires reevaluating global density guarantees at every iterative step. This transforms the convergence process from a linear-time peeling task into a sequence of expensive global optimization problems. The second challenge is the **inapplicability of linear-time decomposition**. Single-layer decomposition relies on the *nesting property* of the scalar density k [15], [16], which allows efficiently retrieving all subgraphs via a peeling-style process. In contrast, the density vector $\mathbf{k}$ lacks a total order, destroying this strict nesting structure. Consequently, there is no single linear ordering to discover all subgraphs. The problem shifts from a linear scan to an *exponential combinatorial search* over feasible density vectors $\mathbf{k}$, rendering exhaustive enumeration infeasible. To tackle these challenges, we propose a suite of efficient algorithms for $D_{\mathbf{k}}$ computation and density decomposition.

For the first challenge, we first propose a single-layer flow-based extraction method (SLFBE algorithm), which utilizes network flow techniques to identify subgraphs with density greater than k_ℓ in each layer ℓ. Building upon this, we introduce a basic search algorithm, that computes $D_{\mathbf{k}}$ for a given density vector $\mathbf{k}$ using an iterative multilayer intersection (IMLI algorithm). However, IMLI suffers from inefficiency. To overcome this, we develop a layer-constrained vertex set propagation framework (LCVSP algorithm), which incorporates a $\mathbf{k}$-core pruning strategy. The $\mathbf{k}$-core pruning efficiently eliminates irrelevant vertices, and the propagation mechanism further reduces the number of iterations, significantly improving overall performance.

For the second challenge, we leverage the structural properties of the *density lattice* to guide the search and reduce unnecessary computations. We first present the vector space hierarchical traversal method (VSHT algorithm), which explores the *density lattice* (Fig. 2) in a breadth-first manner and processes all density vectors at each level. While effective, this approach incurs substantial redundancy. To address this, we propose the hierarchical progressive density decomposition algorithm, HPDD, which incrementally constructs multilayer dense subgraphs by extending partial density vectors layer by layer. This hierarchical strategy enables localized exploration and early pruning, greatly reducing the search space and computational cost. Furthermore, to improve the efficiency and reduce the memory cost of HPDD, we introduce HPDD+, which integrates a recursive depth-first search and a divide-and-conquer approach underpinned by a novel adaptive upper bound. This mechanism adaptively limits the search space, eliminating redundant computations. Besides, this approach reduces the space complexity from exponential to linear in the number of layers (as detailed in Lemma 6).

Our main contributions are summarized as follows.

- We propose a novel density-based model, $\mathbf{k}$-dense subgraph ($D_{\mathbf{k}}$), which guarantees minimum density in each layer.
- We develop LCVSP, an efficient algorithm for computing $D_{\mathbf{k}}$, which combines $\mathbf{k}$-core-based pruning with a layer-constrained vertex set propagation strategy to accelerate convergence and reduce unnecessary computation.
- We advance multilayer density decomposition by proposing a hierarchical progressive approach and developing a efficient HPDD algorithm, along with HPDD+ algorithm, which achieves scalable and memory-optimal computation with linear space complexity.
- We conduct extensive experiments on 9 real-world multilayer graphs, showing that our model consistently discovers higher-quality subgraphs compared to other models, and our algorithms achieve superior efficiency and scalability.

II. Preliminaries

We consider an undirected multilayer graph $G = (V, E, L)$, where V is a set of vertices, L is a set of layers, and $E \subseteq V \times V \times L$ is a set of edges. For a given layer $\ell \in L$, let $E_\ell = \{(u, v, \ell) \in E\}$ be the set of edges on layer ℓ, and let the corresponding layer-specific graph be $G_\ell = (V, E_\ell)$. The degree of a vertex $u \in V$ in layer ℓ is denoted by $d_\ell(u) = |\{(u, v, \ell) \in E_\ell\}|$. Let $\mu(\ell)$ and $\mu(\hat{L})$ denote the minimum degree of a vertex in layer ℓ and over a subset of layers $\hat{L} \subseteq L$, respectively. For a vertex set S, let $\mu(S, \ell)$ and $\mu(S, \hat{L})$ denote the corresponding values in the subgraph induced by S.

Given a node set $S \subseteq V$, the subgraph of G induced by S over all layers is denoted by $G[S] = (S, E[S], L)$,

TABLE I
SUMMARY OF NOTATIONS

Symbol	Description
$G = (V, E, L)$	A multilayer graph with vertex set V, edge set E, layer set L
$G_\ell = (V, E_\ell)$	The subgraph on layer ℓ
$d_\ell(u)$	The degree of vertex u in layer ℓ
$\mathbf{k} = [k_\ell]_{\ell \in L}$	A density vector where k_ℓ is the threshold for layer ℓ
$D_\mathbf{k}$	The multilayer $\mathbf{k}$-dense subgraph
$\vec{G}_\ell$	An orientation of the graph on layer ℓ
$\vec{d}_\ell(u)$	The outdegree of vertex u in oriented graph $\vec{G}_\ell$
$\rho(G)/\rho^*(G)$	The density of graph G / maximum density of graph G
$\delta(G)$	The degeneracy of graph G

where $E[S] = \{(u, v, \ell) \in E : u \in S, v \in S\}$. For a specified subset of layers $\hat{L} = \{1, 2, \ldots, \ell\} \subseteq L$, the induced multilayer subgraph on S restricted to $\hat{L}$ is denoted by $G_{\hat{L}}[S] = (S, E_{\hat{L}}[S], \hat{L})$. For each layer $\ell \in L$, the layer-specific induced subgraph by S is $G_\ell[S] = (S, E_\ell[S])$, and the degree of $u \in S$ in layer ℓ is $d_\ell^S(u)$. For two disjoint vertex sets X and Y satisfying $X \cap Y = \emptyset$, we define the *cross edge* in layer ℓ between X and Y as $E_\times(X, Y, \ell) = \{(x, y) \in E_\ell \mid x \in X,\ y \in Y\}$, capturing edges that connect the two sets.

An orientation of the multilayer graph G is represented as $\vec{G} = (V, \vec{E}, L)$, where each edge in E is assigned a direction. For each layer $\ell \in L$, the corresponding directed graph is $\vec{G}_\ell = (V, \vec{E}_\ell)$, where each directed edge $\langle u, v\rangle \in \vec{E}_\ell$ represents an edge from u to v. The outdegree of a vertex $u \in V$ in layer ℓ is $\vec{d}_\ell(u) = |\{v \mid \langle u, v\rangle \in \vec{E}_\ell\}|$. Similarly, the induced subgraph on layer ℓ with vertex set S is $\vec{G}_\ell[S] = (S, \vec{E}_\ell[S])$, and the outdegree of $u \in S$ within this subgraph is $\vec{d}_\ell^S(u)$. A directed path from s to t in layer ℓ, denoted $s \rightsquigarrow t$, is a sequence of vertices $s = x_0, x_1, \ldots, x_k = t$, such that $\langle x_{i-1}, x_i\rangle \in \vec{E}_\ell$ for all $i = 1, \ldots, k$.

Following the classical definition of density [17], the density of a graph G is defined as $\rho(G) = \frac{|E[G]|}{|V[G]|}$. Let $\rho^*(G)$ denote the density of the densest subgraph within the graph G.

III. THE $D_\mathbf{k}$ SUBGRAPH MODEL

While classical graph density captures cohesiveness in single-layer graphs, real-world networks often involve multiple types of interactions, naturally represented as multilayer graphs. We thus generalize density to this setting via a layer-wise density vector $\mathbf{k} = [k_\ell]_{\ell \in L} \in \mathbb{Z}^{|L|}$, where k_ℓ denotes the minimum density in layer ℓ, leading to the definition of a multilayer dense subgraph (MDS).

Definition 1: (Multilayer Dense Subgraph (MDS) and Density Vector) Given a multilayer graph $G = (V, E, L)$ and a density vector $\mathbf{k} = [k_\ell]_{\ell \in L} \in \mathbb{Z}$, let $\vec{G}$ be an orientation of G, where for each layer $\ell \in L$, the oriented graph is denoted by $\vec{G}_\ell = (V, \vec{E}_\ell)$. For each layer $\ell \in L$, define the vertex sets:

$$S_\ell = \left\{u \in V \mid \vec{d}_\ell(u) > k_\ell\right\}, \quad T_\ell = \left\{u \in V \mid \vec{d}_\ell(u) < k_\ell\right\}.$$

If, for all $\ell \in L$, there exists no path $s \rightsquigarrow t$ in $\vec{G}_\ell$ with $s \in S_\ell$ and $t \in T_\ell$, define the corresponding dense region as $D_\ell = S_\ell \cup \{v \in V \mid \exists u \in S_\ell,\ u \rightsquigarrow v \text{ in } \vec{G}_\ell\}$.

The multilayer $\mathbf{k}$-dense subgraph is defined as the MDS $G[D] = (D \subseteq V, E[D], L)$ such that $D = D_\ell$ for all $\ell \in L$; that is, D induces the same vertex set across all layers.

For brevity, given a density vector $\mathbf{k}$, $D_\mathbf{k}$ refers to either the $\mathbf{k}$-dense subgraph $G[D]$ or its vertex set D. This definition leads to the following observations:

Observation 1: For every $\ell \in L$, the $D_\mathbf{k}$ satisfies:

(1) **Boundary orientation:** All edges between $V \setminus D_\mathbf{k}$ and $D_\mathbf{k}$ in $\vec{G}_\ell$ are oriented to $D_\mathbf{k}$, *e.g.*, $E_\times(D_\mathbf{k}, V \setminus D_\mathbf{k}, \ell) \subseteq \vec{E}_\ell$.

(2) **Minimum outdegree guarantee:** $\forall u \in D_\mathbf{k}$, $\vec{d}_\ell^G(u) = \vec{d}_\ell^{D_\mathbf{k}}(u) \geq k_\ell$.

(3) **External boundedness:** $\forall u \in V \setminus D_\mathbf{k}$, $\vec{d}_\ell^G(u) \leq k_\ell$.

Motivation and design rationale. As summarized in Observation 1, the multilayer $\mathbf{k}$-dense subgraph exhibits a clear separation between vertices inside and outside the dense region. To preserve this equilibrium, our model adopts an *orientation constraint*, which directs all boundary edges inward to ensure local stability, and a *no $S \to T$ path condition*, which prevents weakly connected nodes from invading dense regions. These constraints jointly guarantee that the induced subgraph remains internally dense and externally sparse across all layers. This mechanism guarantees that every vertex in $D_\mathbf{k}$ maintains sufficient connections within the subgraph, which directly leads to a provable density lower bound on each layer (Theorem 1), while preventing weakly connected vertices from being absorbed into the dense region. Consequently, $D_\mathbf{k}$ achieves layer-wise density guarantees and ensures a stable and uniquely defined multilayer structure (Theorem 2).

Theorem 1: Let $G = (V, E, L)$ be a multilayer graph, and let $\mathbf{k} = [k_\ell]_{\ell \in L} \in \mathbb{Z}^{|L|}$ be a density vector. Suppose $D_\mathbf{k}$ is induced by vertex set D according to Definition 1, satisfying $D = D_\ell$ for all $\ell \in L$. Then, for each $\ell \in L$, the following density inequality holds that $\rho(G_\ell[D]) = \frac{|E_\ell[D]|}{|D|} > k_\ell$.

Proof: Fix a layer $\ell \in L$, and consider the orientation $\vec{G}_\ell = (V, \vec{E}_\ell)$. Let $G_\ell[D] = (D, E_\ell[D])$ be the subgraph induced by D. Partition D into two subsets: $S_{k_\ell+1} = \{u \in D : \vec{d}_\ell^S(u) \geq k_\ell + 1\}$, $S_{k_\ell} = \{u \in D : \vec{d}_\ell^S(u) = k_\ell\}$. $|S_{k_\ell+1}| > 0$ as there exists at least one vertex u with $\vec{d}_\ell^D(u) > k_\ell$.

Given that $|S_{k_\ell+1}| > 0$, we can establish:

$$\rho(G_\ell[D]) = \frac{|E_\ell[D]|}{|D|} \geq \frac{(k_\ell + 1)|S_{k_\ell+1}| + (k_\ell)|S_{k_\ell}|}{|S_{k_\ell+1}| + |S_{k_\ell}|} > \frac{(k_\ell)(|S_{k_\ell+1}| + |S_{k_\ell}|)}{|S_{k_\ell+1}| + |S_{k_\ell}|} = k_\ell$$

We can observe that $\rho(G_\ell[D]) > k_\ell$. □

While the density guarantee confirms the tightness of our model, theoretical soundness further requires the uniqueness of the MDS. We now show that each $D_\mathbf{k}$ is uniquely determined by the density vector $\mathbf{k}$.

Theorem 2: Given a multilayer graph G and a density vector $\mathbf{k}$, the $D_\mathbf{k}$ is unique.

Proof: We prove by contradiction. Suppose there exist two distinct $D_\mathbf{k}$s of G, denoted by D_1 and D_2, such that $D_1 \neq D_2$. Define the non-empty set $D = D_1 \setminus D_2$, so that $D \subseteq D_1$ and $D \cap D_2 = \emptyset$. Consequently, we have $|E_\times(D, D_2, \ell)| > |E_\times(D, D_1 \setminus D, \ell)|$.

On the one hand, by the Observation 1 (2), the total outdegree from D in D_1 satisfies $\sum_{u \in D} \vec{d}_\ell^{D_1}(u) \geq k_\ell \cdot |D|$.

To compute the minimum of total consists of the internal edges of D and the cross edges from D to $D_1 \setminus D$, we assume that each vertex $u \in D$ has outdegree exactly k_ℓ in D_1, then by Definition 1, there must exist at least one directed edge from a vertex in $D_1 \setminus D$ to a vertex in D, which implies $|E_\ell[D]|+|E_\times(D, D_1 \setminus D, \ell)| > \sum_{u \in D} \vec{d}_\ell^{G}(u) = \sum_{u \in D} \vec{d}_\ell^{D_1}(u) = k_\ell \cdot |D|$. On the other hand, according to the Observation 1 (1) and (3), it implies that the cross edges from D to D_2 satisfy $|E_\ell[D]| + |E_\times(D, D_2, \ell)| \leq \sum_{u \in D} \vec{d}_\ell^{G}(u) = k_\ell \cdot |D|$. Combining these results, we derive a contradiction: $k_\ell \cdot |D| < |E_\ell[D]| + |E_\times(D, D_1 \setminus D, \ell)| < |E_\ell[D]| + |E_\times(D, D_2, \ell)| \leq k_\ell \cdot |D|$. Therefore, it follows that $D_1 = D_2$, which implies the uniqueness of $D_{\mathbf{k}}$. □

Although each density vector $\mathbf{k}$ uniquely determines a $D_{\mathbf{k}}$, the reverse is not necessarily true. The same vertex set $D \subseteq V$ may satisfy the density conditions for multiple different density vectors. For example, the subgraph induced by vertices $\{v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8\}$ in Fig. 1 forms an MDS under each of the following density vectors, *i.e.*, $[3, 0, 0], [3, 1, 0], [3, 0, 1], [3, 1, 1], [3, 2, 0]$ and $[3, 2, 1]$. Among these, the density vector $[3, 2, 1]$ is not dominated by any other, denoted as the maximal density vector.

Definition 2: (Maximal Density Vector) Let $G = (V, E, L)$ be a multilayer graph, and let $D \subseteq V$ be a vertex set such that $D = D_{\mathbf{k}}$ for some density vector $\mathbf{k} = [k_\ell]_{\ell \in L}$. We say that $\mathbf{k}$ is a *maximal density vector* of D if there does not exist another density vector $\mathbf{k}' = [k'_\ell]_{\ell \in L}$ such that $D = D_{\mathbf{k}'}$, $k'_\ell \geq k_\ell$ for all $\ell \in L$ and $k'_{\hat{\ell}} > k_{\hat{\ell}}$ for some $\hat{\ell} \in L$.

The definition of MDS naturally aligns with the concept of density decomposition in single-layer graph [18]. Inspired by this connection, we aim to extend the notion of density decomposition to multilayer graphs. Formally, we define our problem as follows.

Problem statement: Given a multilayer graph $G = (V, E, L)$, compute the set of all non-empty and distinct $D_{\mathbf{k}}$s of G, each associated with its maximal density vector $\mathbf{k}$. This collection forms the *density decomposition* of the multilayer graph.

Remark: Difference from heterogeneous information networks (HINs). While multilayer graphs are related to HINs [19], they differ in structural assumptions. HINs typically involve multiple types of nodes and edges, focusing on semantic path mining (e.g., meta-paths). In contrast, our multilayer graph model considers a shared vertex set V across layers without inter-layer edges, and aims to identify subgraphs that are simultaneously dense in all layers.

IV. K-Dense Subgraph Search

Given a multilayer graph $G = (V, E, L)$ and a density vector $\mathbf{k}$, we now address the problem of computing the corresponding $D_{\mathbf{k}}$. However, computing the $D_{\mathbf{k}}$ poses two fundamental algorithmic challenges as follows:

(1) Layer-wise subgraph computation. For each layer $\ell \in L$, identifying the subgraph D_ℓ requires satisfying a non-trivial reachability condition: there must be no directed path from any vertex in $S_\ell = \{u \mid \vec{d}_\ell(u) > k_\ell\}$ to any vertex in $T_\ell = \{u \mid \vec{d}_\ell(u) < k_\ell\}$. Naïvely eliminating all $s \rightsquigarrow t$ paths from S_ℓ to T_ℓ is computationally prohibitive, especially in large dense graphs. Thus, the key difficulty is to design a method that can enforce the global path elimination constraint efficiently.

(2) Cross-layer consistency. Even if all layer-specific subgraphs D_ℓ are computed, enforcing that a common vertex set D satisfies $D = D_\ell$ for all $\ell \in L$ is non-trivial. Due to structural differences across layers and varying density thresholds, the sets D_ℓ may not align. Ensuring consistency requires iterative refinement across layers: removing a vertex from one layer may invalidate the density constraint in another, triggering further updates. Designing a convergence-guaranteed refinement procedure with minimal recomputation is critical for scalability.

To address the above challenges, we first propose the SLFBE algorithm, which transforms the $s \rightsquigarrow t$ path elimination requirement into a maximum flow problem, enabling efficient extraction of dense subgraphs on each layer. To enforce cross-layer consistency, we first introduce the IMLI framework, which iteratively applies SLFBE on all layers and refines the candidate vertex set via intersection. To further enhance efficiency, we develop the LCVSP algorithm, which accelerates the computation of $D_{\mathbf{k}}$ by combining multilayer $\mathbf{k}$-core pruning (Theorem 3) with a layer-constrained vertex set propagation strategy.

A. Single-Layer Flow-Based Extraction

To efficiently compute $D_{\mathbf{k}}$, we first extract the dense subgraph D_ℓ from each individual layer $\ell \in L$. This single-layer extraction serves as the computational backbone of both the IMLI and LCVSP frameworks, since $D_{\mathbf{k}} = D_\ell$ is derived from these layer-wise results.

Here, we propose the SLFBE algorithm, which efficiently identifies all edges that need to be reversed in order to eliminate any reachability from S_ℓ to T_ℓ. Instead of iteratively removing individual $s \rightsquigarrow t$ paths, SLFBE reformulates the reachability constraint as a single maximum flow problem over a flow network. The key insight is that the set of saturated edges in this network directly corresponds to the set of edges in the original graph whose reversal will eliminate all $S_\ell \to T_\ell$ reachability. That is, instead of tracking and reversing specific paths, the algorithm computes all necessary edge reversals in one shot by solving a flow problem. This approach is based on the re-orientation network framework [20]. Below, we present how to construct the re-orientation network for a single layer of the multilayer graph.

Definition 3: (Re-orientation Flow Network) [20] For an orientation graph $\vec{G}_\ell = (V, \vec{E}_\ell)$ and a positive integer k_ℓ, we construct a flow network $\vec{G}_\ell{}' = (V', \vec{E}_\ell{}', c)$ on the ℓ-th layer graph. Here, $V' = V \cup \{s, t\}$ where s is the source and t is the sink. The edge set $\vec{E}_\ell{}'$ includes three types of edges with non-negative capacities $c(e)$: (1) for each $\langle u, v\rangle$ in $\vec{G}_\ell$ with $c(u, v) = 1$; (2) for each u in V where $\vec{d}_\ell(u) > k_\ell$, an edge $\langle s, u\rangle$ with $c(s, u) = \vec{d}_\ell(u) - k_\ell$; and (3) for each u in V where $\vec{d}_\ell(u) < k_\ell$, an edge $\langle u, t\rangle$ with $c(u, t) = k_\ell - \vec{d}_\ell(u)$.

The complete procedure is shown in Algorithm 1. It first constructs the re-orientation network, then computes the max-

Algorithm 1: SLFBE

Input: A single layer graph $\vec{G}_\ell = (V, \vec{E}_\ell)$ and an integer k_ℓ.
Output: k_ℓ-dense subgraph.

$V' \leftarrow V \cup \{s, t\}$; // Add source and sink
for *each vertex* $v \in V$ **do**
  if $\vec{d}_\ell(v) > k_\ell$ **then**
    Add arc $\langle s, v\rangle$ to the $\vec{E}_\ell{}'$ with $c(s, v) = \vec{d}_\ell(v) - k_\ell$;
  if $\vec{d}_\ell(v) < k_\ell$ **then**
    Add arc $\langle v, t\rangle$ to the $\vec{E}_\ell{}'$ with $c(v, t) = k_\ell - \vec{d}_\ell(v)$;
for *each* $\langle u, v\rangle \in \vec{E}_\ell$ **do**
  Add arc $\langle u, v\rangle$ to the $\vec{E}_\ell{}'$ with $c(u, v) = 1$;
Compute the maximum flow from s to t of $\vec{G}_\ell{}' = (V', \vec{E}_\ell{}', c)$;
return $D_\ell \leftarrow$ All vertices reachable from vertex s in the residual network;

Algorithm 2: IMLI

Input: A multilayer graph $\vec{G} = (V, \vec{E}, L)$ and an integer vector $\mathbf{k} = [k_\ell]_{\ell \in L} \in \mathbb{Z}$.
Output: $D_{\mathbf{k}}$.

1 $C \leftarrow V$; $changed \leftarrow$ True;
2 **while** $changed$ **do**
3 $changed \leftarrow$ False; $C_{old} \leftarrow C$;
4 **for** *each layer* $\ell \in L$ **do**
5 $D_\ell \leftarrow$ SLFBE($\vec{G}_\ell[C], k_\ell$);
6 $C \leftarrow \bigcap_{\ell \in L} D_\ell$;
7 **if** $C = \emptyset$ **then return** $\emptyset$;
8 **if** $C \neq C_{old}$ **then**
9 $C \leftarrow C_{old}$; $changed \leftarrow$ True;
10 **return** $G[C] = (C, E[C], L)$;

imum flow. Once the maximum flow is computed, the resulting minimum s-t cut guarantees that no path remains from S_ℓ to T_ℓ, effectively satisfying the reachability condition defined for D_ℓ. The set of nodes reachable from s in the residual graph forms the desired subgraph. The following theorem shows the correctness and complexity of Algorithm 1.

Lemma 1: The SLFBE algorithm can output D_ℓ correctly with a time complexity of $O(|E_\ell|^{1.5})$ and a space complexity of $O(|E_\ell|)$.

Proof: We compute the maximum flow in $\vec{G}_\ell{}'$, by the max-flow min-cut theorem, there is no augmenting path remains. In particular, all directed paths from any node in S_ℓ to any node in T_ℓ must be "cut" by saturated arcs. Suppose for each *saturated* edge $\langle x, y\rangle \in \vec{E}_\ell{}'$ in $\vec{G}_\ell{}'$, it is reversed in $\vec{G}_\ell{}'$. Thus the saturated-arc reversals eliminate all such $s \rightsquigarrow t$ paths. Then in $\vec{G}_\ell{}'$, the set S_ℓ will contain only the nodes in the residual network connected to s by unsaturated edges, and the set T_ℓ will contain only the nodes in the residual network connected to t by unsaturated edges. Because there is no path $s \rightsquigarrow t$ in the residual network, there is also no path from the set S_ℓ to the set T_ℓ in $\vec{G}_\ell{}'$. By Definition 1, the reachability closure of S_ℓ exactly yields the unique k_ℓ-dense subgraph D_ℓ.

The time and space complexity of the maximum flow computation on G_ℓ is $O(|E_\ell|^{1.5})$ and $O(|E_\ell|)$ respectively [21]. □

Remark: Orientation invariance. Although Algorithm 1 initializes the flow network using an arbitrary orientation $\vec{G}_\ell$, the resulting dense subgraph D_ℓ is independent of this choice. The max-flow computation implicitly determines the necessary edge re-orientations to satisfy the density constraints. As guaranteed by Theorem 2, any initial orientation leads to the same unique dense subgraph, rendering the decomposition framework orientation-invariant for the original undirected multilayer graph.

B. Iterative Multi-Layer Intersection

After solving the subproblem of extracting each D_ℓ from individual layers, the main challenge is to consolidate these layer-specific subgraphs into a single unified $D_{\mathbf{k}}$. To this end, we introduce the IMLI algorithm, as shown in Algorithm 2. The algorithm initializes the candidate set with all vertices (line 1), and iteratively refines it until convergence. At each iteration, it invokes SLFBE on the subgraph induced by the current candidate set in each layer (lines 4–5), and computes the intersection of the resulting subgraphs (line 6). It checks whether the candidate set has changed compared to the previous iteration and continues only if further updates are required (lines 8–9). The process terminates when the candidate set C stabilizes.

Lemma 2: The IMLI framework achieves a time complexity of $O(t \cdot |L| \cdot \sum_{\ell \in L} |E_\ell|^{1.5})$ and a space complexity of $O(\sum_{\ell \in L} |E_\ell|)$, where t is the number of iterations until convergence, $|L|$ is the number of layers, and $|E_\ell|$ denotes the number of edges in layer ℓ.

Proof: The total cost contains two aspects. First, in each iteration, the algorithm applies SLFBE to each layer, which takes $O(|E_\ell|^{1.5})$ per layer. Summing over all layers and iterations gives the time complexity $O(t \cdot |L| \cdot \sum_{\ell \in L} |E_\ell|^{1.5})$. Second, the intersection and candidate update operations are bounded by $O(|V|)$ per iteration, which is dominated by the flow computation. □

C. Layer-Constrained Vertex Set Propagation

As mentioned, although the IMLI algorithm ensures cross-layer consistency through iterative intersection, it suffers from inefficiencies. Each layer is processed independently, leading to redundant computations. Specifically, vertices that are already excluded by one layer may still be unnecessarily processed in other layers, which not only increases computational overhead but also delays overall convergence.

To overcome this limitation, we propose a more efficient framework, LCVSP, which introduces a layer-constrained vertex set propagation mechanism. Instead of recomputing on the entire candidate set in each layer, LCVSP processes layers sequentially: the result of SLFBE on one layer is propagated as the input subgraph to the next layer. This progressive refinement avoids considering vertices already filtered out by earlier layers, effectively reducing redundant computation and improving convergence efficiency.

Moreover, to further accelerate computation, LCVSP incorporates a multilayer $\mathbf{k}$-core pruning strategy. Based on the Theorem 3, we preemptively eliminate vertices that cannot belong to the final $D_{\mathbf{k}}$, thereby significantly narrowing the search space. The combined design of cross-layer vertex set propagation and $\mathbf{k}$-core-based pruning makes LCVSP both

Algorithm 3: LCVSP

Input: A multilayer graph $\vec{G} = (V, \vec{E}, L)$ and an integer vector $\mathbf{k} = [k_\ell]_{\ell \in L} \in \mathbb{Z}$.
Output: $D_{\mathbf{k}}$

1 $C \leftarrow V$; $changed \leftarrow$ True;
2 **while** $\exists u \in C, \exists \ell \in L : d_\ell^C(u) < k_\ell + 1$ **do**
3 $\quad C \leftarrow C \setminus \{u\}$;
4 **while** $changed$ **do**
5 $\quad changed \leftarrow$ False;
6 $\quad$ **for** *each layer* $\ell \in L$ **do**
7 $\quad\quad$ **if** $G_\ell[C]$ *is empty* **then**
8 $\quad\quad\quad$ **return** $\emptyset$;
9 $\quad\quad D \leftarrow$ SLFBE($\vec{G_\ell}[C], k_\ell$) ; // Algorithm 1
10 $\quad\quad$ **if** $D \neq C$ **then**
11 $\quad\quad\quad C \leftarrow D$; $changed \leftarrow$ True;
12 **return** $G[C] = (C, E[C], L)$;

theoretically sound and practically scalable for large multilayer graphs. Next, we introduce the concept of multilayer $\mathbf{k}$-core.

Definition 4: (Multilayer $\mathbf{k}$-core) [10], [22] Given a multilayer graph $G = (V, E, L)$ and an $|L|$-dimensional integer vector $\mathbf{k} = [k_\ell]_{\ell \in L}$, the multilayer $\mathbf{k}$-core of G is a maximal subgraph $G[C] = (C \subseteq V, E[C], L)$ such that $\forall \ell \in L : \mu(C, \ell) \geq k_\ell$.

We observe that the multilayer $\mathbf{k}$-core is closely related to the target $D_{\mathbf{k}}$. This relationship is formalized below:

Theorem 3: Let $G = (V, E, L)$ be a multilayer graph and $\mathbf{k} = [k_\ell]_{\ell \in L}$ be a density vector. Let $C_{\mathbf{k}'}$ denote the multilayer core where $\mathbf{k}' = \mathbf{k} + \mathbf{1}$ (i.e., $k'_\ell = k_\ell + 1, \forall \ell \in L$). Then, $D_{\mathbf{k}} \subseteq C_{\mathbf{k}'} = C_{\mathbf{k+1}}$.

Proof: Fix a layer $\ell \in L$, and consider the $\vec{G_\ell} = (V, \vec{E_\ell})$ of the ℓ-th layer. This trivially holds when $D_{\mathbf{k}} = \emptyset$, so we focus on case where $D_{\mathbf{k}} \neq \emptyset$. By Observation 1, every node $u \in D_{\mathbf{k}}$ satisfies $\vec{d}_\ell^{D_{\mathbf{k}}}(u) \geq k_\ell$. In particular, if $\vec{d}_\ell^{D_{\mathbf{k}}}(u) = k_\ell$, then u must have at least one incoming edge in $\vec{G}(D_{\mathbf{k}})$. Otherwise, by Definition 1, node u would not qualify for inclusion in $D_{\mathbf{k}}$, contradicting our assumption. Since the total degree of a node in $\vec{G_\ell}$ equals the sum of its indegree and outdegree in the orientation $\vec{G_\ell}$, we conclude that $d_\ell^{D_{\mathbf{k}}}(u) \geq k_\ell + 1$. Therefore, $D_{\mathbf{k}} \subseteq C_{\mathbf{k+1}}$ □

This inclusion result provides a powerful pruning rule: any vertex not in $C_{\mathbf{k}+1}$ is guaranteed not to appear in $D_{\mathbf{k}}$. By applying this result, we can significantly narrow the initial search space before invoking costly flow-based computations.

The LCVSP algorithm equipped with such core reduction technique is depicted in Algorithm 3. LCVSP begins by extracting $C_{\mathbf{k+1}}$ to initialize the candidate set C (lines 1-3). Then, the key idea of LCVSP lies in its layer-constrained vertex set propagation strategy. Specifically, in each iteration (lines 4–11), the algorithm sequentially processes each layer $\ell \in L$, applying the SLFBE algorithm on the subgraph $G_\ell[C]$ induced by C (line 7). This step enforces the layer-specific density constraint k_ℓ, ensuring that only vertices meeting the requirement in layer ℓ are retained. If pruning occurs in any layer (*i.e.*, $D \neq C$), the updated set $C \leftarrow D$ is immediately propagated to subsequent layers in the same iteration (lines 9–11). The process terminates once C stabilizes and satisfies the density requirements across all layers.

Although the worst-case time and space complexity of LCVSP matches that of IMLI, namely $O(t \cdot |L| \cdot \sum_{\ell \in L} |E_\ell|^{1.5})$ and $O(\max_{\ell \in L} |E_\ell|)$ respectively, LCVSP achieves notable empirical speedups. This improvement stems from two sources: the $(\mathbf{k} + \mathbf{1})$-core pruning eliminates non-qualifying vertices early, and the layer-constrained propagation strategy avoids repeated evaluation of irrelevant candidates. Together, these enhancements make LCVSP substantially more efficient in practice while maintaining correctness and scalability.

V. Multilayer Density Decomposition

Density decomposition of multilayer graphs is computationally challenging due to the combinatorial explosion of candidate density vectors. A naive exhaustive search would explore a space of size $O(\prod_{\ell=1}^{L} \lceil \rho^*(G_\ell) \rceil)$. To address this, we propose a structural approach based on the concept of a *density lattice*, illustrated in Fig. 2. This structure takes the form of a directed acyclic graph (DAG), where each node represents a feasible density vector $\mathbf{k}$ with non-empty $D_{\mathbf{k}}$, and directed edges encode unit-wise increments: there is an edge from $\mathbf{k}$ to $\mathbf{k}'$ if $\mathbf{k}' = \mathbf{k} + \mathbf{e}_\ell$ for some $\ell \in L$. Each node $D_{\mathbf{k}}$ is treated as the parent of its immediate successors $D_{\mathbf{k}'}$.

To efficiently traverse the *density lattice*, we propose three algorithms that systematically enumerate these subgraphs. First, we present the VSHT algorithm, which employs a breadth-first search strategy to explore the *density lattice* level by level. This approach enumerates and processes all complete density vectors $\mathbf{k} \in \mathbb{Z}_{\geq 0}^L$, but it suffers from redundant computations. To overcome this, we propose a novel HPDD algorithm. It adopts a layer-wise decomposition strategy, progressively constructing the decomposition hierarchy in a layer-by-layer manner; Finally, to further enhance scalability by minimizing memory consumption, we propose an advanced algorithm, denoted as HPDD+, that combines recursive enumeration with divide-and-conquer techniques to achieve both space efficiency and reduced computational redundancy.

A. Vector Space Hierarchical Traversal

The VSHT algorithm performs a level by level traversal of the *density lattice* , building on the following theorem.

Theorem 4: Let $G = (V, E, L)$ be a multilayer graph. For density vectors $\mathbf{k}, \mathbf{k}' \in \mathbb{Z}_{\geq 0}^L$, let $D_{\mathbf{k}}$ and $D_{\mathbf{k}'}$ denote the corresponding MDSs. If $\mathbf{k}' \leq \mathbf{k}$ (i.e., $k'_\ell \leq k_\ell$ for all $\ell \in L$), then $D_{\mathbf{k}} \subseteq D_{\mathbf{k}'}$.

Proof: We proceed by contradiction. Assume, for the sake of contradiction, that $D_{\mathbf{k}} \not\subseteq D_{\mathbf{k}'}$, implying that the set $D = D_{\mathbf{k}} \setminus D_{\mathbf{k}'}$ is non-empty (i.e., $D \neq \emptyset$). According to Theorem 2, the induced subgraph on D in layer ℓ must satisfy the density condition: $|E_\ell(D)| + |E_\times(D, D_{\mathbf{k}} \setminus D, \ell)| > k_\ell \cdot |D|, |E_\ell(D)| + |E_\times(D, D_{\mathbf{k}'}, \ell)| \leq k'_\ell \cdot |D|$. Because $D = D_{\mathbf{k}} \setminus D_{\mathbf{k}'}$, all vertices in D are not in $D_{\mathbf{k}'}$. Therefore, $D_{\mathbf{k}} \setminus D \subseteq D_{\mathbf{k}'}$. As a result, $|E_\times(D, D_{\mathbf{k}'}, \ell)| \geq |E_\times(D, D_{\mathbf{k}} \setminus D, \ell)|$. Combining these inequalities and the condition $k'_\ell \leq k_\ell$, we derive the contradiction $k_\ell \cdot |D| < |E(D)| + |E_\times(D, D_{\mathbf{k}} \setminus D, \ell)| \leq |E(D)| + |E_\times(D, D_{\mathbf{k}'}, \ell)| \leq k'_\ell \cdot |D|$. Thus, $D_{\mathbf{k}} \subseteq D_{\mathbf{k}'}$. □

Algorithm 4: VSHT

Input: A multilayer graph $G = (V, E, L)$.
Output: The set $\mathcal{D}$ of all non-empty MDSs of G.

1 $\mathcal{D} \leftarrow \emptyset$, $Q \leftarrow \{[0]_{|L|}\}$, $\mathcal{P}([0]_{|L|}) \leftarrow \emptyset$; // Track parent nodes
2 **while** $Q \neq \emptyset$ **do**
3 dequeue density vector $\mathbf{k}$ from Q;
4 $P_n \leftarrow \bigcap_{F \in \mathcal{P}(\mathbf{k})} F$; // Lemma 3
5 Arbitrarily orient the subgraph $G[P_n]$ to obtain $\vec{G}[P_n]$;
6 $\mathcal{D}_k \leftarrow$ LCVSP($\vec{G}[P_n], \mathbf{k}$) ; // Algorithm 3
7 **if** $\mathcal{D}_k \neq \emptyset$ **then**
8 $\mathcal{D} \leftarrow \mathcal{D} \cup \{\mathcal{D}_k\}$;
9 **for** *each* $\ell \in L$ **do**
10 $\mathbf{k}' \leftarrow [k_1, \ldots, k_\ell + 1, \ldots, k_{|L|}]$;
11 enqueue $\mathbf{k}'$ into Q;
12 $\mathcal{P}(\mathbf{k}') \leftarrow \mathcal{P}(\mathbf{k}') \cup \{\mathcal{C}_k\}$;
13 **return** $\mathcal{D}$;

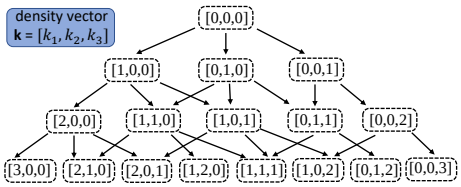


Fig. 2. A three-layer graph density decomposition organizes all $D_{\mathbf{k}}$s into a *density lattice* , revealing the evolution of cohesive patterns across layers.

This containment property enables pruning in the *density lattice*. In particular, any $D_{\mathbf{k}}$ must be fully contained within the intersection of its parent subgraphs in the *density lattice*. This yields the following lemma:

Lemma 3: Given a multilayer graph G and a density vector $\mathbf{k}$, let $\mathcal{P}(D_{\mathbf{k}})$ be the set of parents of $D_{\mathbf{k}}$ in the *density lattice* of G. It holds that $D_{\mathbf{k}} \subseteq \bigcap_{\hat{D} \in \mathcal{P}(D_{\mathbf{k}})} \hat{D}$.

Proof: By definition of *density lattice*, the density vector of all parents $\mathcal{P}(D_{\mathbf{k}})$ of $D_{\mathbf{k}}$ is dominated by the $\mathbf{k}$. Thus, according to Theorem 4, it holds that $D_{\mathbf{k}} \subseteq D_{\mathbf{k}'}$, $\forall D_{\mathbf{k}'} \in \mathcal{P}(D_{\mathbf{k}})$. Assume a vertex $u \notin \bigcap_{\hat{D} \in \mathcal{P}(D_{\mathbf{k}})} \hat{D}$, $u \in D_{\mathbf{k}}$ exists. This implies that there exists a parent $D' \in \mathcal{P}(D_{\mathbf{k}})$ such that $D_{\mathbf{k}} \nsubseteq D'$, thus leading to a contradiction. □

The VSHT algorithm, as showned in Algorithm 4, implements these theoretical insights through a breadth-first search strategy that systematically traverses the density vector space. Starting with the root density vector $[0]_{|L|}$ (line 1), it maintains a queue of density vectors, dequeuing and processing each density vector $\mathbf{k}$ by computing its $D_{\mathbf{k}}$ through Algorithm 3 on the intersection of parent nodes (line 6). Non-empty $D_{\mathbf{k}}$ are added to the result set, and child density vectors are generated by incrementing individual layer density (lines 10-12).

Lemma 4: Let B be the number of density vectors visited in the *density lattice* . The total time complexity is $O(B \cdot t \cdot |L| \cdot \sum_{\ell \in L} |E_\ell|^{1.5})$. In practice, B is much smaller than the theoretical upper bound $O\left(\prod_{\ell=1}^{|L|} \rho^*(G_\ell)\right)$, as the algorithm uses parent intersection to restrict subgraph size and prunes branches leading to empty subgraphs.

For space complexity, VSHT only maintains subgraphs with active children in the queue, requiring at most two levels of the *density lattice* . Thus, the space usage is $O(|V| \cdot W)$, where W is the maximum width of two consecutive lattice levels.

B. Hierarchical Progressive Density Decomposition

In this subsection, we introduce the HPDD algorithm. Unlike VSHT, which traverses the density vector space by evaluating each density vector $\mathbf{k} \in \mathbb{Z}_{\geq 0}^L$ defined over all layers, HPDD incrementally constructs the density vector $\mathbf{k}$ by progressively extending partial density vectors one layer at a time. This hierarchical progressive strategy enables localized exploration and early pruning, thereby significantly reducing the search space.

To formalize this hierarchical process, we first define the concept of a partial density vector and its extension.

Definition 5 (Partial Density Vector): Let $G = (V, E, L)$ be a multilayer graph. For any j with $1 \leq j < |L|$, a *partial density vector* over the first j layers is defined as $\mathbf{k}^j = [k_1, \ldots, k_j]$.

Based on this, the search proceeds by extending a valid vector from the current layer to the next.

Definition 6 (Partial Density Vector Extension): Given a partial density vector $\mathbf{k}^j = [k_1, \ldots, k_j]$, we define its *extension* by a density threshold $i \in \mathbb{Z}_{\geq 0}$ at layer $j+1$ as the new vector: $\mathbf{k}^{(j+1)} = \mathbf{k}^j \circ i = [k_1, \ldots, k_j, i]$.

Monotonic pruning (analogous to Apriori). The efficiency of HPDD relies on a fundamental containment property: for any extension $\mathbf{k}^{(j+1)}$ derived from $\mathbf{k}^j$, the corresponding dense subgraph must be a subset of the subgraph induced by the prefix, i.e., $D_{\mathbf{k}^{(j+1)}} \subseteq D_{\mathbf{k}^j}$. This implies a powerful pruning rule: if a partial density vector $\mathbf{k}^j$ yields an empty subgraph ($D_{\mathbf{k}^j} = \emptyset$), then any of its extensions to subsequent layers will inevitably be empty.

This logic mirrors the *downward closure property* found in frequent itemset mining algorithms like Apriori [23]. By leveraging this monotonicity, HPDD safely prunes entire branches of the search space as soon as a density constraint is violated in a prefix layer, avoiding unnecessary computations in higher dimensions.

Based on this property, we formulate the recursive enumeration strategy:

Theorem 5: For a multilayer graph $G = (V, E, L)$ with $\ell = |L|$ layers, all non-empty $D_{\mathbf{k}}$ can be enumerated through the recursive procedure: (1) Compute all non-empty $D_{\mathbf{k}^1}$ on the first layer with $\mathbf{k}^1 = [k_1]$. (2) For each non-empty $D_{\mathbf{k}^j}$, compute all non-empty $D_{\mathbf{k}^{(j+1)}}$ induced by $D_{\mathbf{k}^j}$ on the first $j+1$ layers, where $\mathbf{k}^{(j+1)} = [k_1, k_2, \ldots, k_j, k_{j+1}]$. (3) Continue this process until all ℓ layers are considered.

Proof: We prove this by induction over the layers.

Base case: For a single-layer graph, computing all non-empty $D_{\mathbf{k}^1}$, where $\mathbf{k}^1 = [k_1]$, enumerates all possible MDSs.

Inductive hypothesis: Assume the theorem holds for any multilayer graph with j layers, where $1 \leq j < \ell$.

Inductive step: Consider a non-empty $D_{\mathbf{k}^{(j+1)}}$ defined on the first $j+1$ layers with vector $\mathbf{k}^{(j+1)} = \mathbf{k}^j \circ k_{j+1}$. By the definition of density constraints, $D_{\mathbf{k}^{(j+1)}}$ must satisfy the constraints of the prefix vector $\mathbf{k}^j$. This implies that $D_{\mathbf{k}^{(j+1)}} \subseteq D_{\mathbf{k}^j}$. Since $D_{\mathbf{k}^{(j+1)}} \neq \emptyset$, it follows that $D_{\mathbf{k}^j} \neq \emptyset$. By the inductive hypothesis, $D_{\mathbf{k}^j}$ has been enumerated in

Algorithm 5: HPDD

Input: A multilayer graph $G = (V, E, L)$.
Output: The set $\mathcal{D}$ of all non-empty MDSs of G.

1 $\mathcal{D} \leftarrow \emptyset$, $Q \leftarrow \emptyset$, enqueue $(V, \emptyset)$ into Q;
2 **for** $\ell \leftarrow 1$ **to** $|L|$ **do**
3 $\quad Q' \leftarrow \emptyset$;
4 $\quad$ **while** $Q \neq \emptyset$ **do**
5 $\quad\quad$ dequeue (S, K) from Q;
6 $\quad\quad \hat{L} \leftarrow \{1, 2, \ldots, \ell\}$;
7 $\quad\quad G'[S] \leftarrow (S, E[S], \hat{L})$;
8 $\quad\quad$ Arbitrarily orient the subgraph $G'[S]$ to obtain $\vec{G}'[S]$;
9 $\quad\quad$ **for** *each* $(u, \ell) \in S \times \hat{L}$ **do**
10 $\quad\quad\quad \vec{d}_\ell(u) \leftarrow$ outdegree of u in $\vec{G}'_\ell[S]$;
11 $\quad\quad$ enqueue $(S, K \circ 0)$ into Q';
12 $\quad\quad$ **for** $i \leftarrow 1, 2, 3 \ldots$ **do**
13 $\quad\quad\quad \mathbf{k} \leftarrow K \circ i$;
14 $\quad\quad\quad D_{\mathbf{k}} \leftarrow$ LCVSP$(\vec{G}'[S], \mathbf{k})$;
15 $\quad\quad\quad$ **if** $D_{\mathbf{k}} \neq \emptyset$ **then**
16 $\quad\quad\quad\quad$ enqueue $(D_{\mathbf{k}}, \mathbf{k})$ into Q';
17 $\quad\quad\quad\quad$ **if** $\ell = |L|$ **then** $\mathcal{D} \leftarrow \mathcal{D} \cup D_{\mathbf{k}}$;
18 $\quad\quad\quad$ **else break**;
19 $\quad Q \leftarrow Q'$;

the previous step. The algorithm explicitly computes all non-empty subgraphs induced by $D_{\mathbf{k}^j}$ for all valid extensions k_{j+1}. Consequently, $D_{\mathbf{k}^{(j+1)}}$ is guaranteed to be enumerated. □

The pseudocode of HPDD is presented in Algorithm 5. HPDD begins with the entire vertex set and an empty density vector (line 1), and iteratively processes layers from $\ell = 1$ to $|L|$ (lines 2–19). At each layer ℓ, HPDD considers all candidate pairs $(S, \mathbf{k}^{(\ell-1)})$ in Q, where S is the vertex set corresponding to a valid partial density vector up to layer $\ell-1$. For each such pair (lines 4–18), the algorithm constructs a multilayer subgraph $G'[S]$ induced by S over the first ℓ layers (lines 6–7), and computes an orientation (lines 8–10).

It then extends $\mathbf{k}^{(\ell-1)}$ by incrementing the ℓ-th component (lines 11–18). For each candidate $\mathbf{k}^\ell = \mathbf{k}^{(\ell-1)} \circ i$, the LCVSP algorithm is applied to $\vec{G}'[S]$ to compute the corresponding $D_{\mathbf{k}^\ell}$. If the result is non-empty, the pair $(D_{\mathbf{k}^\ell}, \mathbf{k}^\ell)$ is enqueued into Q' for further exploration. If $\ell = |L|$, the result is added to the $\mathcal{D}$. Otherwise, if an empty subgraph is encountered, further increments are pruned.

To concretely demonstrate how HPDD operates, we consider the example shown in Fig. 3.

Example 1: ①: Start with layer 1 by constructing $G_{\hat{L}} = (V, E_{\hat{L}}, \hat{L})$ and computing all $\mathbf{k}^1 = [k_1]$ with $k_1 \in \{0, 1, 2, 3\}$ and corresponding MDSs $D_{\mathbf{k}^1}$. ②: For each $D_{\mathbf{k}^1}$, the algorithm constructs $G_{\hat{L}} = (D_{\mathbf{k}^1}, E_{\hat{L}}, \hat{L})$ with $\hat{L} = \{1, 2\}$. Then compute all $\mathbf{k}^2 = [k_1, k_2]$ with valid k_2 values (e.g., $k_2 \in \{0, 1\}$ when $k_1 = 0$). ③: For each $D_{\mathbf{k}^2}$, further extend to layer $\{3\}$ to obtain $\mathbf{k}^3 = [k_1, k_2, k_3]$; e.g., for $\mathbf{k}^2 = [0, 1]$, compute $k_3 \in \{0, 1, 2\}$.

Lemma 5: The worst-case time complexity of the Algorithm 5 is identical to that of the Algorithm 4, i.e., $O(B \cdot t \cdot |L| \cdot \sum_{\ell \in L} |E_\ell|^{1.5})$. This bound arises because, in the worst case, HPDD may enumerate all possible non-empty density vectors. However, due to the hierarchical progressive strategy, the number of explored subgraph candidates at each stage is

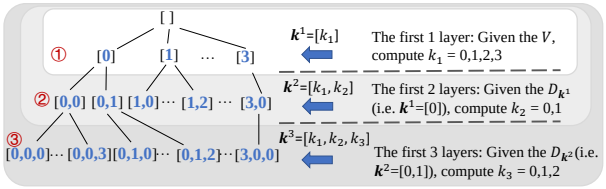


Fig. 3. An example of HPDD for 3 layer graph

significantly reduced. As a result, the practical value of t is much smaller for HPDD than for VSHT.

The space complexity of the Algorithm 5 is bounded by $O(|V| \cdot \prod_{\ell=1}^{L} \lceil \rho^*(G_\ell) \rceil)$. However, in real-world multilayer graphs, the space complexity is often significantly smaller than its theoretical bound. This is because it is generally impossible for a vertex subset to simultaneously achieve the maximum density on all layers. Consequently, the actual number of non-empty candidates is much lower than the worst-case bound.

Proof: Let us consider the first ℓ layers, the set of all partial density vectors as K^ℓ, with $|K^\ell| \leq \prod_{j=1}^{\ell} \lceil \rho^*(G_\ell) \rceil$. For each density vector $\mathbf{k}^\ell \in K^\ell$, the corresponding candidate subgraph $D_{\mathbf{k}^\ell}$ is stored explicitly.

We prove by induction on ℓ:

Base case ($\ell = 1$): At the first layer, the number of partial density vectors is at most $\rho^*(G_1)$, each with a candidate subgraph of size at most $|V|$. So, total space: $O(|V| \cdot \rho^*(G_1))$.

Inductive step: Assume that at layer ℓ, the number of non-empty candidates is at most $\prod_{j=1}^{\ell} \lceil \rho^*(G_j) \rceil$. At layer $\ell + 1$, for each such candidate, the number of possible extensions is at most $\lceil \rho^*(G_{\ell+1}) \rceil$. Thus, at layer $\ell + 1$, the number of non-empty candidates is at most: $|K^{(\ell+1)}| \leq |K^\ell| \cdot \lceil \rho^*(G_{\ell+1}) \rceil \leq \prod_{j=1}^{\ell+1} \lceil \rho^*(G_j) \rceil$. Each associated subgraph requires at most $|V|$ space. By induction, after all $|L|$ layers, the total space required is: $O\left(|V| \cdot \prod_{\ell=1}^{|L|} \lceil \rho^*(G_\ell) \rceil\right)$. □

C. The Space Efficient Algorithm: HPDD+

While HPDD improves the efficiency of multilayer density decomposition, it is still hindered by two critical limitations on large-scale graphs: (1) excessive memory consumption due to the need to maintain all intermediate MDSs corresponding to partial density vectors, and (2) redundant computation arising from the linear enumeration of density thresholds. Since dense subgraphs satisfy a nesting property, vertices in high-density regions are repetitively re-evaluated during the computation of every preceding lower-density threshold at each layer, leading to significant wasted effort. To address these bottlenecks, we propose HPDD+, an advanced algorithm that combines a recursive depth-first search with a divide-and-conquer technique, achieving both space efficiency and computational efficiency.

Space optimization via DFS. Instead of the layer-by-layer traversal used in HPDD, HPDD+ adopts a recursive DFS strategy. For each valid partial density vector $\mathbf{k}^j$, it immediately recursively computes extensions to layer $j+1$ through $|L|$ before backtracking. This strategy drastically reduces space complexity. Instead of storing the entire frontier of partial solutions, HPDD+ only maintains the subgraphs along the current recursion stack (depth $|L|$). The space complexity

Algorithm 6: HPDD+

Input: A multilayer graph $G = (V, E, L)$.
Output: The set $\mathcal{D}$ of all non-empty MDSs of G.

1 $\mathcal{D} \leftarrow \emptyset$; $\mathbf{k}^0 \leftarrow []$;
2 `Recursive`$(G, V, [0], 0, 1)$;
3 **Function** `Recursive`$(G, C, \mathbf{k}^\ell, ub, \ell)$:
4 $\quad Q \leftarrow \emptyset$;
5 $\quad$ **if** $k_\ell = 0$ **then**
6 $\quad\quad ub \leftarrow \delta(G_\ell[C])$; `// Core decomposition [16]`
7 $\quad l \leftarrow 0, R^l_\ell \leftarrow C$; $u \leftarrow ub, R^u_\ell \leftarrow \emptyset$;
8 $\quad$ Divide-and-Conquer $(\vec{G}, \mathbf{k}^\ell, \ell, R^l_\ell, R^u_\ell)$;
9 $\quad$ **for** $i \leftarrow 0$ **to** u **do**
10 $\quad\quad$ **if** $R^i_\ell \neq \emptyset$ **then**
11 $\quad\quad\quad$ **if** $\ell = |L|$ **then**
12 $\quad\quad\quad\quad \mathbf{k} \leftarrow \mathbf{k}^\ell$; $D_\mathbf{k} \leftarrow R^i_\ell$; $\mathcal{D} \leftarrow \mathcal{D} \cup D_\mathbf{k}$;
13 $\quad\quad\quad$ **else**
14 $\quad\quad\quad\quad ub \leftarrow i$; enqueue $(R^i_\ell, \mathbf{k}^{(\ell-1)} \circ i)$ into Q;
15 $\quad ub' \leftarrow ub$; `// Preserve upper bound before recursion`
16 $\quad$ **if** $\ell + 1 > |L|$ **then**
17 $\quad\quad$ **for** *each pair* $(D_{\mathbf{k}^\ell}, \mathbf{k}^\ell)$ *in* $(\mathcal{C}, \mathcal{K})$ **do**
18 $\quad\quad\quad \hat{L} \leftarrow \{1, 2, \ldots, \ell, \ell+1\}$;
19 $\quad\quad\quad G'[D_{\mathbf{k}^\ell}] \leftarrow (D_{\mathbf{k}^\ell}, E[D_{\mathbf{k}^\ell}], \hat{L})$;
20 $\quad\quad\quad$ Arbitrarily orient the subgraph $G'[S]$ to obtain $\vec{G}'[S]$;
21 $\quad\quad\quad$ **for** *each* $(u, \ell') \in S \times \hat{L}$ **do**
22 $\quad\quad\quad\quad \vec{d}_{\ell'}(u) \leftarrow$ outdegree of u in $\vec{G}'_{\ell'}[S]$;
23 $\quad\quad\quad \mathbf{k}^{\ell+1} \leftarrow \mathbf{k}^\ell \circ 0$;
24 $\quad\quad\quad$ `Recursive`$(G'[D_{\mathbf{k}^\ell}], D_{\mathbf{k}^\ell}, \mathbf{k}^{\ell+1}, ub, \ell+1)$;
25 $\quad ub \leftarrow ub'$;

is reduced from $O(|V| \cdot \prod \lceil \rho^* \rceil)$ to $O(|V| \cdot \sum \lceil \rho^* \rceil)$ (see Lemma 6), making it feasible for massive graphs.

Divide-and-Conquer optimization. Within each recursive step at layer ℓ, we must identify all valid density thresholds k_ℓ that yield non-empty subgraphs. A naive linear scan is inefficient. We observe that dense subgraphs exhibit a *hierarchical containment property*: for a fixed prefix of partial density vector, increasing k_ℓ results in a smaller subgraph, i.e., $D_{\mathbf{k} \circ y} \subseteq D_{\mathbf{k} \circ x}$ for $y > x$. Based on this, we derive the *Locally Resolvable Property*, which enables a divide-and-conquer approach.

Theorem 6 (Locally Resolvable Property): Let $\mathbf{x}^\ell$ and $\mathbf{y}^\ell$ be two partial density vectors differing only at layer ℓ with $x_\ell < y_\ell$. For any intermediate threshold $p_\ell \in (x_\ell, y_\ell)$, the corresponding dense subgraph $D_{\mathbf{p}^\ell}$ can be computed as: $D_{\mathbf{p}^\ell} = \mathsf{LCVSP}\left(\vec{G}[D_{\mathbf{x}^\ell} \setminus D_{\mathbf{y}^\ell}],\ p_\ell\right) \cup D_{\mathbf{y}^\ell}$.

Proof: The containment property implies $D_{\mathbf{y}^\ell} \subseteq D_{\mathbf{p}^\ell} \subseteq D_{\mathbf{x}^\ell}$. Thus, $D_{\mathbf{p}^\ell}$ decomposes into a "known" core $D_{\mathbf{y}^\ell}$ and an "unknown" fringe $D_{\mathbf{p}^\ell} \setminus D_{\mathbf{y}^\ell}$. Crucially, due to the boundary orientation (Observation 1), no edges point from the core $D_{\mathbf{y}^\ell}$ to the fringe. Therefore, the density condition for vertices in the fringe depends solely on the induced subgraph $\vec{G}[D_{\mathbf{x}^\ell} \setminus D_{\mathbf{y}^\ell}]$. The union follows immediately. □

This resolution process, illustrated in Fig. 4, efficiently computes any intermediate subgraph $D_{\mathbf{p}^\ell}$ once the MDSs corresponding to $\mathbf{x}^\ell$ and $\mathbf{y}^\ell$ are known. The recursion then proceeds independently on the two disjoint regions—$D_{\mathbf{x}^\ell} \setminus D_{\mathbf{p}^\ell}$ and $D_{\mathbf{p}^\ell} \setminus D_{\mathbf{y}^\ell}$, enabling reuse of previous results and progressive search space reduction.

The Theorem 6 guarantees that if we know the subgraphs

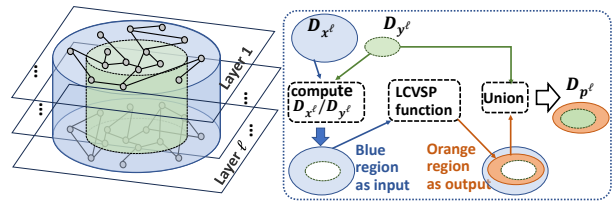


Fig. 4. Illustration of the locally resolvable property

for a lower bound l and an upper bound u, we can efficiently compute the subgraph for the midpoint mid by operating only on the residual nodes $D_{\mathbf{k} \circ l} \setminus D_{\mathbf{k} \circ u}$, rather than the entire graph.

Adaptive upper bound. To effectively apply the divide-and-conquer strategy, we must first define a tight search interval $[lb, ub]$ for the density threshold k_ℓ. While the lower bound is naturally 0 (inheriting the parent subgraph), the upper bound can be loosely constrained by graph degeneracy. However, we can derive a tighter *adaptive upper bound* by leveraging the history of the recursive traversal.

Theorem 7: For any partial density vector $\mathbf{k}^j$, the maximum valid density threshold at layer $j+1$ is bounded by: $ub_{j+1} \leq \min\{\max_{\mathbf{p}^j \prec \mathbf{k}^j} \triangle_{j+1}(\mathbf{p}^j), \delta(G_{j+1}[D_{\mathbf{k}^j}])\}$ where $\delta(G_{j+1}[D_{\mathbf{k}^j}])$ is the degeneracy of the induced subgraph on layer $j+1$, $\mathbf{p}^j \prec \mathbf{k}^j$ denotes vectors preceding $\mathbf{k}^j$ in the recursive order, and $\triangle$ is the maximum observed density.

Proof: The adaptive upper bound is derived from the intersection of two constraints:

(1) Structural Bound: By the property established in [24], the density of any subgraph is bounded by the degeneracy of the underlying graph. Applying this to the induced subgraph on layer $j+1$, the maximum attainable density for any extension of $D_{\mathbf{k}^j}$ is bounded by $\delta(G_{j+1}[D_{\mathbf{k}^j}])$.

(2) Recursive History Bound: Consider the recursive nature of the algorithm. Let $\mathbf{p}^j$ be a partial density vector preceding $\mathbf{k}^j$ in the enumeration order such that $D_{\mathbf{k}^j} \subseteq D_{\mathbf{p}^j}$. Since the subgraph induced by $D_{\mathbf{k}^j}$ on layer $j+1$ is a subset of that induced by $D_{\mathbf{p}^j}$, the maximum density achievable starting from $D_{\mathbf{k}^j}$ cannot exceed the maximum density $\triangle_{j+1}(\mathbf{p}^j)$ already observed for $D_{\mathbf{p}^j}$.

The algorithm safely prunes the search space by taking the minimum of these two bounds:
$ub_{j+1} \leq \min\{\max_{\mathbf{p}^j \prec \mathbf{k}^j} \triangle_{j+1}(\mathbf{p}^j), \delta(G_{j+1}[D_{\mathbf{k}^j}])\}$. □

Based on the above results, we develop the HPDD+ algorithm, presented in Algorithm 6, and the recursive divide-and-conquer subroutine is detailed in Algorithm 7.

The HPDD+ algorithm initiates the recursive enumeration by invoking the `Recursive` function (line 2), starting from the initial vector $[0]$ on layer $\ell = 1$. Each recursive call computes all valid extensions of a given partial density vector $\mathbf{k}^\ell = [k_1, k_2, \ldots, k_\ell]$ (lines 3–25). At each layer ℓ, it maintains the feasible interval $[0, ub]$ of k_ℓ values, where the upper bound ub is either inherited or computed via core decomposition if $k_\ell = 0$ (line 5). Given this interval, the algorithm invokes the Divide-and-Conquer subroutine (line 7) to compute all $R^i_\ell(D_{\mathbf{k}^\ell})$ for $i \in [0, ub]$.

If a valid R^i_ℓ is found, the algorithm either outputs it (when $\ell = |L|$), or stores it for further expansion to the next layer.

Algorithm 7: Divide-and-Conquer

Input: A multilayer graph $\vec{G} = (V, \vec{E}, L)$, $\mathbf{k}^\ell$, ℓ, R_ℓ^u, R_ℓ^l.
Output: $\mathbf{k}$-dense subgraph.

1 **if** $u - l \leq 1$ ***or*** $R_\ell^u = R_\ell^l$ **then**
2   **if** $R_\ell^u = R_\ell^l$ **then**
3   **for** $i \leftarrow l + 1$ **to** $u - 1$ **do** $\quad R_\ell^i \leftarrow R_\ell^l$;
4   **return**;
5 $mid \leftarrow \lceil (u + l + 1)/2 \rceil$;
6 $R \leftarrow R_\ell^l \setminus R_\ell^u$;
7 **for** *each* $\ell \in L$ **do**
8   **for** *each* $(u, v) \in E_\times(R, R_\ell^u, \ell)$ **do**
9   **if** $\langle v, u \rangle \wedge (u \in R) \wedge (v \in R_\ell^u))$ **then**
10   reorient edge as $\langle u, v \rangle$;
11             $\vec{d}_\ell(u) \leftarrow \vec{d}_\ell(u) + 1$;
12 $\mathbf{k}^\ell \leftarrow \mathbf{k}^{\ell-1} \circ mid$;
13 $R_\ell^{mid} \leftarrow$ LCVSP$(\vec{G}[R], \mathbf{k}^\ell) \cup R_\ell^u$;   // Algorithm 3
14 **if** $R_\ell^{mid} \neq \emptyset$ **then**
15   Divide-and-Conquer $(\vec{G}, \mathbf{k}^\ell, \ell, R_\ell^{mid}, R_\ell^l)$;
16   Divide-and-Conquer $(\vec{G}, \mathbf{k}^\ell, \ell, R_\ell^u, R_\ell^{mid+1})$;

For each such subgraph, the algorithm constructs the induced multilayer subgraph and orients edges to support local density constraint checking in the next recursive call (lines 15-23). The process continues until all layers have been processed.

The divide-and-conquer subroutine (Algorithm 7) computes all feasible values of k_ℓ. It recursively splits the interval $[l, u]$, applies LCVSP on the residual region $R = R_\ell^l \setminus R_\ell^u$, and merges results with R_ℓ^u to obtain R_ℓ^{mid} (line 12). Once R_ℓ^{mid} is computed, the algorithm proceeds to explore both halves of the current interval.

Lemma 6: The worst-case time complexity of the Algorithm 6 is $O(B \cdot t \cdot |L| \cdot \sum_{\ell \in L} |E_\ell|^{1.5})$. The space complexity of the Algorithm 5 is bounded by $O(|V| \cdot \sum_{\ell=1}^{L} \lceil \rho^*(G_\ell) \rceil)$.

Proof: In terms of space complexity, HPDD+ only needs to store dense subgraphs along the current recursive path. The maximum recursion depth is $|L|$, and at each layer ℓ, we store at most $O(\lceil \rho^*(G_\ell) \rceil)$ subgraphs. Therefore, the space complexity is $O(|V| \cdot \sum_{\ell=1}^{L} \lceil \rho^*(G_\ell) \rceil)$, which is substantially more efficient for large multilayer networks. □

Discussions. Parallelization strategy: our framework naturally supports hierarchical parallelization. At the lattice level, the recursive structure of HPDD+ yields independent search branches, enabling coarse-grained task parallelism. At the layer level, while LCVSP requires iterative coordination, the bottleneck layer-wise density checks (e.g., max-flow) are decoupled within each iteration, allowing for fine-grained concurrent execution. This two-tier strategy ensures effective scalability on multi-core architectures.

Practical guide. Practitioners should interpret density vectors as structural profiles for targeted discovery. Balanced vectors (e.g., $\mathbf{k} = [13, 14, 14]$ as validated in Exp-9) imply robust cross-layer consensus, which is ideal for identifying cohesive units like biological functional modules. In contrast, skewed vectors (e.g., $\mathbf{k} = [8, 4, 4]$ in Exp-10) highlight layer-specific anomalies, effectively isolating heterogeneous structures such as hidden fraud rings. Furthermore, exploring incremental variations along the *density lattice* enables distinguishing stable core‘ members from the periphery, facilitating granular analysis of node roles within communities.

VI. Experiments

A. Experimental Setup

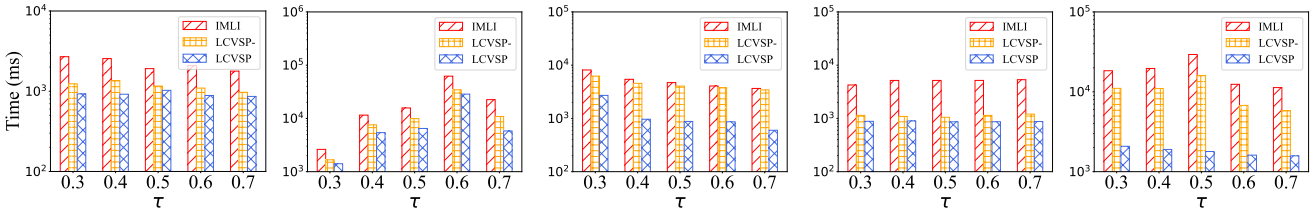
Algorithms. To evaluate our proposed multilayer $\mathbf{k}$-dense subgraph model, we implement algorithms for two key tasks: (1) computing $D_\mathbf{k}$ for a given density vector $\mathbf{k}$, and performing density decomposition over all feasible $\mathbf{k}$. Specifically, we implement IMLI (Algorithm 2) and LCVSP (Algorithm 3) for computing a $D_\mathbf{k}$, and three decomposition algorithms: VSHT (Algorithm 4), HPDD (Algorithm 5), and HPDD+ (Algorithm 6). Since our formulation of $\mathbf{k}$-dense subgraph is novel and tailored to multilayer graphs, no existing methods directly support $D_\mathbf{k}$ computation or complete decomposition. As baselines, we therefore use simplified versions of our own algorithms: a basic IMLI algorithm for $D_\mathbf{k}$ computation, and the VSHT algorithm for density decomposition. During decomposition, $D_\mathbf{k}$ computation is repeatedly invoked as a subroutine, where we employ LCVSP for optimal performance. All algorithms are implemented in C++ and compiled with the g++ compiler. Experiments are conducted on a Linux system equipped with an Intel Xeon Gold 5218R CPU and 96GB DDR4 memory.

Datasets. We evaluate our methods on 9 real-world multilayer graphs, summarized in Table II. The Homo and SacchCere datasets are genetic interaction networks [25]; Cannes2013 [26], Higgs [25], ObamaInIsrael [25], FriendFeed [27], and FlickrGrowth [28] are social networks, where FlickrGrowth is transformed into a multilayer graph by grouping edges by timestamp. Finally, DBLP[1] and Amazon[2] represent collaboration networks from bibliographic and product co-purchasing domains, respectively.

B. Efficiency Evaluation

Exp-1: Runtime of $\mathbf{k}$-dense subgraph search algorithms. To evaluate the efficiency of our proposed algorithms, we compare LCVSP (Algorithm 3), LCVSP- (*i.e.*,LCVSP without $\mathbf{k}$-core pruning) and IMLI (Algorithm 2) on five large multilayer graphs, by varying the threshold parameter $\tau \in \{0.3, 0.4, 0.5, 0.6, 0.7\}$. For each value of τ, the corresponding density vector $\mathbf{k}$ is set as $k_\ell = \lfloor \tau \cdot \rho^*(G_\ell) \rfloor$. As shown in Fig. 5, IMLI generally incurs higher runtime. In contrast, LCVSP- significantly outperforms IMLI on most datasets, benefiting from the vertex set propagation technique that reduces redundant computation across layers. Building upon this, LCVSP further improves efficiency by incorporating an initial $\mathbf{k}$-core pruning, which eliminates globally unpromising vertices. For example, on the FG dataset, LCVSP completes in 1,789 ms, while IMLI takes 29,330 ms—resulting in a 16.39× speed-up. Overall, LCVSP achieves the best runtime performance among all methods.

[1]https://dblp.uni-trier.de/xml/
[2]http://snap.stanford.edu/data/



(a) OI (b) AM (c) HI (d) FF (e) FG

Fig. 5. Runtime of different **k**-dense subgraph search algorithms (vary τ)

TABLE II
DATASETS USED IN THE EXPERIMENTS

Graphs	Abbr.	Category	$\|V\|$	$\|E\|$	$\|L\|$	$\min_{\ell \in L} \frac{\|E_\ell\|}{\|V\|}$	$\max_{\ell \in L} \frac{\|E_\ell\|}{\|V\|}$
Homo	HO	Genetic	18k	153k	7	0.014	4.583
SacchCere	SC	Genetic	6.5k	247k	7	0.205	13.997
Cannes2013	CA	Social	438k	962k	3	0.179	1.121
DBLP	DB	Co-authorship	513k	1.0M	10	0.187	0.220
ObamaInIsrael	OI	Social	2.2M	3.8M	3	0.244	0.792
Amazon	AM	Co-purchasing	410k	8.1M	4	2.193	5.956
Higgs	HI	Social	456k	13M	4	0.061	27.392
FriendFeed	FF	Social	505k	18M	3	0.527	35.669
FlickrGrowth	FG	Social	2.3M	23M	2	2.139	6.686

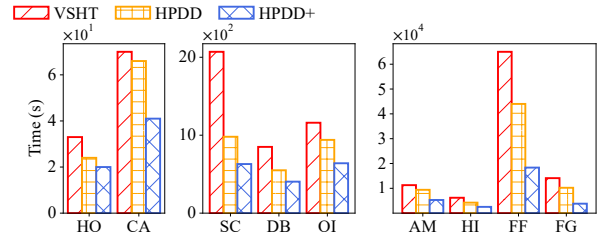


Fig. 6. Runtime of density decomposition algorithms

Regarding the influence of density thresholds, we observe that runtime generally decreases as τ increases on most datasets, as higher density requirements enable the **k**-core pruning to aggressively reduce the search space. A notable exception is the AM dataset. At low $\tau = 0.3$, constraints are loose and the algorithm converges almost instantly with minimal vertex removal. At $\tau = 0.6$, the graph enters a dense regime where structures become sensitive to perturbations, inducing long chains of cascading updates and thus prolonging convergence compared to lower or higher thresholds.

Exp-2: Runtime of various density decomposition algorithms. We compare the runtime of three density decomposition algorithms. As illustrated in Fig. 6, VSHT exhibits the highest runtime across all datasets. HPDD and HPDD+ consistently outperform the baseline VSHT algorithm across all datasets, with more pronounced speedup on larger and denser graphs such as FF and FG datasets. The HPDD algorithm achieves a 1.5× to 2× speedup over VSHT on most datasets, validating the effectiveness of its hierarchical progressive pruning strategy in reducing unnecessary computation. HPDD+ further accelerates the process using a divide-and-conquer approach, achieving up to a 4× speedup on the FF dataset. These empirical findings are consistent with theoretical expectations, as the enumeration of $D_{\mathbf{k}}$ remains the core computational bottleneck. These results underscore that advanced pruning and progressive decomposition strategies,

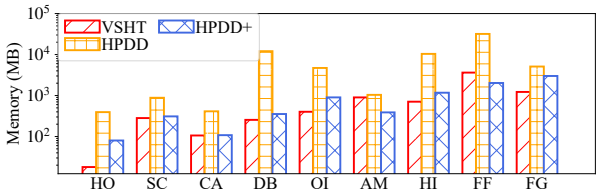


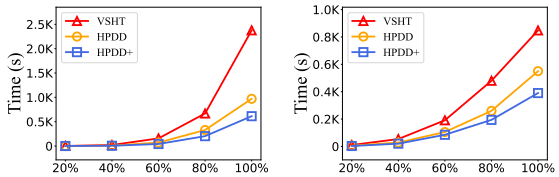
Fig. 7. Memory costs of density decomposition algorithms

even when the worst-case complexity remains unchanged, offer tangible benefits for multilayer dense subgraph discovery.

Exp-3: Memory costs of density decomposition algorithms. In terms of memory usage of our proposed algorithms, the results are depicted in Fig. 7. VSHT consistently consumes the least memory among most datasets. In contrast, HPDD incurs the highest memory overhead across all datasets. The reason is that it requires storing a large number of intermediate subgraphs and density vector states during computation, which are consistent with the theoretical analysis of the space complexity of the algorithm. For instance, on the FF dataset, HPDD's memory cost exceeds 30GB, while VSHT and HPDD+ remain below 3GB. Despite the memory efficiency of VSHT, it suffers from long runtimes as previously shown. In this regard, HPDD+ strikes a balance: it achieves a runtime that is significantly better than both VSHT and HPDD, while maintaining memory cost comparable to VSHT. This demonstrates the advantage of the divide-and-conquer technique, HPDD+ also provides a memory-efficient solution for density decomposition.

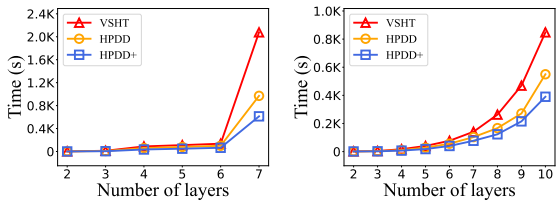
Exp-4: Scalability tests. First, we study how the algorithms scale with the number of vertices. To this end, we randomly sample 20% to 80% of vertices from the original multilayer graphs (SC and DB), and measure runtime on the induced subgraphs. Actually, similar findings are observed on the remaining datasets as well. As shown in Fig. 8, all algorithms require more running time as the vertex set $|V|$ increase. However, both HPDD and HPDD+ consistently outperform VSHT, and the gap widens on larger graphs. Notably, the runtime incurred by HPDD+ shows a steady increase, whereas VSHT demonstrates a substantially steeper increase in runtime. This trend is consistent across datasets with different vertex scales, highlighting the superior scalability and efficiency of HPDD+.

Second, we evaluate scalability with respect to the number of layers. Specifically, we incrementally construct multilayer graphs by selecting the first ℓ layers ($2 \leq \ell \leq |L|$) from the SC and DB datasets. Fig. 9 illustrates that the runtime of



(a) SC (b) DB

Fig. 8. Scalability tests (vary $|V|$)



(a) SC (b) DB

Fig. 9. Runtime of density decomposition algorithms w.r.t. the number of layers in the input multilayer graph.

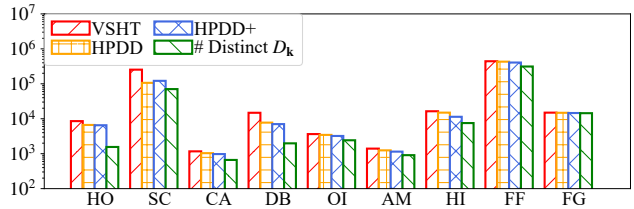
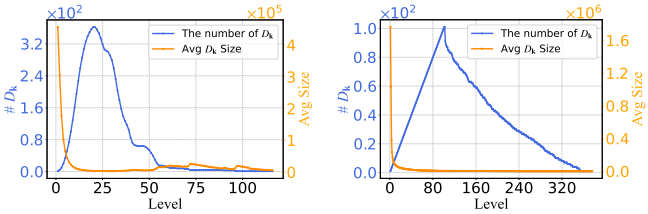


Fig. 10. Comparison between computed and actual distinct $D_{\mathbf{k}}$ across algorithms.



(a) HI (b) FG

Fig. 11. Level-wise distribution of the number and average size of $D_{\mathbf{k}}$s in the density lattice

all algorithms grows rapidly with the increase in the number of layers, reflecting the exponential complexity associated with multilayer dense subgraph enumeration. Interestingly, a sharp runtime increase is observed for all methods when transitioning from 6 to 7 layers on the SC dataset, due to increased cohesiveness in $G_7[V]$. Despite this, HPDD and especially HPDD+ exhibit significantly better scalability. The runtime gap between VSHT and the two advanced algorithms becomes pronounced as the number of layers increases.

C. Density Decomposition Characterization

Exp-5: Comparison between computed and actual $D_{\mathbf{k}}$s. To evaluate the redundancy in the density decomposition, we compare the number of $D_{\mathbf{k}}$ subgraphs computed by three algorithms with the actual number of distinct and non-empty subgraphs. As shown in Fig. 10, for all datasets, the number of computed $D_{\mathbf{k}}$s significantly exceeds the actual count. This gap reflects two intrinsic issues, (1) many density vectors $\mathbf{k}$ lead to empty subgraphs, and (2) multiple non-maximal density vectors may induce the same vertex set, as captured by the Definition 2. Among all methods, VSHT consistently produces the largest number of redundant results, while HPDD shows moderate improvement. Notably, HPDD+ yields computed subgraph counts that closely match the ground truth across datasets. This observation aligns with earlier experiments where HPDD+ demonstrated the highest efficiency.

Exp-6: Distribution of $D_{\mathbf{k}}$ along the density lattice. We analyze the structural characteristics of the density lattice by examining the distribution of $D_{\mathbf{k}}$s across lattice levels. As seen in Fig. 11, despite the different datasets, both HI and FG exhibit consistent trends. The number of $D_{\mathbf{k}}$s per level grows rapidly and reaches its peak within the first 25–30% of the lattice depth, where the density vectors contain mostly small entries (*i.e.*, k_ℓ). After reaching the peak, the number of $D_{\mathbf{k}}$s gradually declines as the density constraints become more restrictive. In contrast, the average size of $D_{\mathbf{k}}$s follows an inverse trend. $D_{\mathbf{k}}$s at the lower levels contain a large portion of the original nodes, while those at the higher levels shrink to less than 1% of the initial size, often comprising a few nodes.

D. Effectiveness Evaluation

We evaluate the effectiveness of the proposed $D_{\mathbf{k}}$ model by comparing it with 5 representative multilayer cohesive subgraph models, namely $\mathbf{k}$-core [10], [22], d-CC [11], [29], FirmCore [12], FocusCore [13], and FirmTruss [14].

Exp-7: Quality of subgraphs discovered by different models. To further assess the effectiveness of the $D_{\mathbf{k}}$ model beyond density, we evaluate the quality of the densest subgraphs (with $\beta = 1$) discovered by each method using two widely-used metrics: the *global clustering coefficient* (GCC) [30] and *conductance* (CDT) [31]. GCC measures the internal connectivity within a subgraph. For each layer $\ell \in L$, $\text{GCC}_\ell = \frac{3\times\text{\# triangles in } G_\ell[D]}{\text{\# connected triples in } G_\ell[D]}$. A higher GCC indicates a more cohesive structure. CDT measures separation from the rest of the graph: $\text{CDT}_\ell = \frac{|E_\ell(D, V\setminus D)|}{\sum_{v\in D} d_\ell(v)}$. Lower CDT values imply stronger structural separation. We report the minimum, maximum, and average values of each metric across layers (e.g., $\text{GCC}_{\min} = \min_{\ell\in L}\text{GCC}_\ell$, $\text{GCC}_{\max} = \max_{\ell\in L}\text{GCC}_\ell$, $\text{GCC}_{\text{avg}} = \frac{1}{|L|}\sum_{\ell\in L}\text{GCC}_\ell$, and similarly for CDT)

The results are summarized in Table III. Across all datasets, $D_{\mathbf{k}}$ consistently achieves the highest average GCC and lowest average CDT across datasets, indicating that its subgraphs are both cohesive and well-separated. While some baselines (e.g., FirmTruss) may excel in individual layers, they often show imbalance across layers. In contrast, $D_{\mathbf{k}}$ maintains strong performance on both metrics, demonstrating its ability to discover well-rounded and structurally robust subgraphs.

Exp-8: Comparison with single-layer baselines. To evaluate the limitations of applying single-layer algorithms to multilayer graphs, we compare $D_{\mathbf{k}}$ against two "Independent-then-Intersect" baselines: INDP-MaxCore and INDP-Densest. These methods independently compute the maximal k-core

TABLE III

PERFORMANCE COMPARISON OF DIFFERENT METRICS ACROSS VARIOUS DATASETS. (METRICS: I: $\mathrm{GCC}_{\min}$, II: $\mathrm{GCC}_{\max}$, III: GCC_{avg}, IV: $\mathrm{CDT}_{\min}$, V: $\mathrm{CDT}_{\max}$, VI: CDT_{avg})

Dataset	SC						OI						HI					
Metric	i	ii	iii	iv	v	vi	i	ii	iii	iv	v	vi	i	ii	iii	iv	v	vi
$\mathbf{k}$-core	0.11	0.38	0.21	0.54	0.87	0.75	0.03	0.12	0.08	0.90	0.91	0.91	0.01	0.14	0.05	0.83	0.95	0.9
FirmCore	0.09	0.23	0.14	**0.35**	**0.60**	**0.46**	0.03	0.11	0.07	0.87	0.91	0.89	0.01	0.14	0.05	0.83	0.95	0.9
d-CC	0.11	0.38	0.21	0.54	0.87	0.75	0.02	0.11	0.07	0.89	0.89	0.89	0.01	0.14	0.05	0.83	0.95	0.9
FocusCore	0.11	0.38	0.21	0.54	0.87	0.75	0.02	0.11	0.07	0.89	0.89	0.89	0.01	0.14	0.05	0.83	0.95	0.9
FirmTruss	0.03	**0.77**	0.23	0.84	0.98	0.94	0.04	0.12	0.10	0.91	0.92	0.92	0.02	**0.35**	0.13	0.87	0.96	0.9
$D_{\mathbf{k}}$	**0.18**	0.52	**0.31**	0.37	0.77	0.61	**0.05**	**0.14**	**0.11**	**0.81**	**0.83**	**0.82**	**0.02**	0.22	**0.15**	**0.70**	**0.90**	**0.8**

TABLE IV

COMPARISON WITH SINGLE-LAYER INTERSECTION BASELINES. REPORT THE SUBGRAPH SIZE ($|V|$) AND THE LAYER-WISE DENSITY.

Dataset	Method	Result Density	Size ($\|V\|$)	Improv.
SC	INDP-MaxCore	Empty Intersection	0	-
	INDP-Densest	Empty Intersection	0	-
	$D_{[1,1,1,1,1,1,1]}$	**[4.6,2.5,2.3,4.3,1.2,1.1,3.9]**	**152**	∞
OI	INDP-MaxCore	[14.5, 10.1, 3.9]	158	-
	INDP-Densest	[16.7, 12.3, 4.9]	259	-
	$D_{[16,12,2]}$	**[22.2,14.7,5.2]**	**495**	**1.91**$\times$
HI	INDP-MaxCore	[6.9, 5.1, 4.7, 0.6]	21	-
	INDP-Densest	[7.2, 4.5, 5.1, 1.2]	23	-
	$D_{[7,4,5,1]}$	**[12.3,7.5,7.5,1.7]**	**93**	**4.04**$\times$

(with maximum coreness) or the densest subgraph on each layer, respectively, and then intersect the resulting vertex sets. For fairness, we adopt an output-aligned protocol where the input density vector $\mathbf{k}$ for our model is calibrated to the actual layer-wise density ($\boldsymbol{\rho}_{base}$) of the INDP-Densest result (i.e., $\mathbf{k} \approx \lfloor \boldsymbol{\rho}_{base} \rfloor$). This ensures that any improvement in subgraph size is achieved under similar or higher density guarantees.

The results in Table IV reveal the fundamental limitations of the independent intersection strategy. On SC, both baselines return an empty set, indicating that locally dense regions are often spatially disjoint across layers (structural mismatch), rendering simple intersection ineffective. On HI and OI, the baselines suffer from premature pruning, where nodes failing a local threshold in a single weak layer are discarded entirely, causing a chain reaction that fragments the global structure. In contrast, by enforcing layer-wise density constraints globally, our model effectively coordinates the structural cohesiveness across layers. This allows it to preserve critical "bridge" nodes and recover subgraphs that are significantly larger (up to $4\times$) while maintaining strictly higher density in every layer compared to the baselines.

E. Case Studies.

Exp-9: Functional system discovery in biological networks. To validate our density definition as a latent feature, we analyze the *Saccharomyces cerevisiae* dataset (Physical, Genetic, Co-expression layers) [32], [33]. We compare the $D_{\mathbf{k}}$ ($\mathbf{k} = [13, 14, 14]$) against the $\mathbf{k}$-core ($\mathbf{k} = [16, 16, 16]$). For a rigorous comparison, we configure all other models to their maximal non-empty settings with strict cross-layer cohesiveness; notably, baselines including d-CC, FirmCore, and FocusCore converge to the same vertex set as the $\mathbf{k}$-core. Experimental results demonstrate that $D_{\mathbf{k}}$ consistently outperforms the $\mathbf{k}$-core in edge density across all three layers,

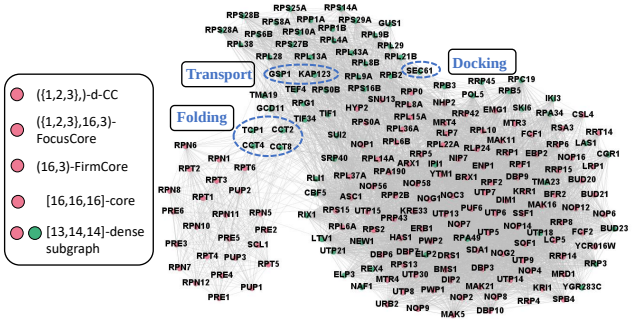


Fig. 12. Functional system discovery in Saccharomyces cerevisiae dataset. Node labels represent protein names. While $\mathbf{k}$-core is limited to the static ribosomal core (pink), $D_{\mathbf{k}}$ (pink + green) recovers the complete dynamic system, including Transport, Docking, and Folding modules.

achieving densities of [23.7, 16.5, 34.0] compared to the $\mathbf{k}$-core's [21.7, 15.3, 27.9].

Qualitative analysis. Fig. 12 (Co-expression layer) illustrates the topological and functional differences between the models. The $\mathbf{k}$-core identifies a compact "static core" of 129 ribosomal subunits. In contrast, $D_{\mathbf{k}}$ discovers a more comprehensive "dynamic functional system" (196 nodes). By enforcing layer-wise density, $D_{\mathbf{k}}$ successfully recruits critical modules that are structurally coupled but topologically peripheral: (1) *Transport Machinery*; (2) *Functional Docking*; and (3) *Folding Coordination*. This confirms that $D_{\mathbf{k}}$ captures the complete functional workflow rather than just the high-degree center.

Quantitative evaluation. We quantify this capability by evaluating the retrieval performance against the "Ground Truth" system (union of Biogenesis, Transport, and Folding functions). As shown in Table V, $D_{\mathbf{k}}$ defies the typical precision-recall trade-off. Despite expanding the community size by 67 nodes, $D_{\mathbf{k}}$ achieves a higher Precision (+5.3%) than $\mathbf{k}$-core. More importantly, it demonstrates a 60.0% improvement in Global Recall and a 44.6% increase in F1-Score. These results provide strong evidence that the proposed density abstraction serves as a superior structural prior for function prediction tasks, effectively propagating labels to biologically integral but peripheral nodes.

TABLE V

PERFORMANCE ON RIBOSOMAL FUNCTIONAL SYSTEM RETRIEVAL

Model	Precision	Recall	F1-Score	True Positives
$\mathbf{k}$-core	0.6589	0.1700	0.2703	129
$D_{\mathbf{k}}$	**0.6939**	**0.2720**	**0.3908**	**196**
Improvement	*+5.3%*	*+60.0%*	*+44.6%*	*+67 nodes*

Exp-10: Fraud Detection in Financial Networks. We evaluate the effectiveness of our model on the AMLSim benchmark [34], [35], a standard simulator for generating anti-money-laundering (AML) transaction graphs. In this dataset, each alert signifies a group of transactions matching a known fraud pattern, with all involved accounts labeled as fraudulent. We construct a three-layer graph (Transfers, Attributes, Co-occurrence). We configure the density vector $\mathbf{k}$ to capture the heterogeneity of fraud patterns: high density in the transfer layer and valid support in auxiliary layers. Specifically, we

fix the auxiliary thresholds to their maximum density ($k_2 = k_3 = 4$) and maximize the transfer layer threshold, resulting in $\mathbf{k} = [8, 4, 4]$ for $D_{\mathbf{k}}$. For a fair comparison, we exhaustively searched for the maximal cohesive subgraphs for all baselines under the same cross-layer support requirements. All baselines converged to a uniform multilayer $\mathbf{k}$-core with $\mathbf{k} = [6, 6, 6]$ as their best achievable structure; increasing thresholds further for baselines resulted in empty or trivial outputs.

The results highlight the superiority of the density-based abstraction. The $D_{\mathbf{k}}$ model identifies a community of 734 users containing 125 fraudsters, achieving a precision of **17.03%** and recall of **16.85%**. In contrast, $\mathbf{k}$-core retrieves a smaller group (662 users) with only 85 fraudsters, yielding significantly lower precision (**12.84%**) and recall (**11.45%**). By enforcing global density constraints, $D_{\mathbf{k}}$ successfully retains these high-value, heterogeneous fraud patterns, improving Precision by 32.63% over the baselines.

VII. Related work

Cohesive Subgraph Models in Multilayer Graphs. Prior studies on multilayer graphs have primarily focused on extracting structurally cohesive subgraphs. Jethava *et al.* [36] formulated the densest common subgraph problem and proposed a linear programming-based solution. Azimi *et al.* [37] introduced the notion of multilayer $\mathbf{k}$-core, which was later extended by Galimberti *et al.* [10], [22] through algorithms for enumerating all valid $\mathbf{k}$-cores. Liu *et al.* [11] studied the CoreCube problem, which explores subgraphs where each node satisfies uniform degree constraints on selected layers—a special case of the general $\mathbf{k}$-core model. Zhu *et al.* [29] addressed the diversification of such subgraphs, while Hashemi *et al.* [12] and Wang *et al.* [13] introduced the FirmCore and FocusCore models to improve robustness and adaptability. Except core-based subgraph decomposition on multilayer graphs, truss-based models offer another direction. Huang *et al.* [38] proposed TrussCube, identifying subgraphs where each edge maintains sufficient support across layers. FirmTruss [14] further generalized this concept by aggregating edge support across multiple layers. While these approaches capture certain forms of cohesiveness, they are not designed to model subgraph density in a strict sense. In contrast, our model enforces explicit density lower bounds for each layer, thereby proving the cohesiveness consistency across all layers.

Dense Subgraph Models. In unipartite graphs, Tatti *et al.* [39] introduced a locally-dense decomposition framework based on densest subgraphs, enabling the identification of all locally dense regions. Danish *et al.* [40] further accelerated this process by leveraging convex optimization techniques. Qin *et al.* [41] proposed the notion of locally densest subgraphs to capture the densest subgraph structures across different graph regions. Borradaile *et al.* [18] formalized density decomposition as a comprehensive framework for dissecting graph density structures. Zhang *et al.* [15] further accelerated this process by using reorientation network. Additionally, Zhang *et al.* [42] also proposed density decomposition for bipartite graphs. While these methods provide valuable insights and tools for dense subgraph analysis, they primarily focus on single-layer graphs and bipartite graphs. To the best of our knowledge, no prior work has extended density decomposition models to multilayer graphs. Our research addresses this gap by proposing a novel density decomposition framework tailored for multilayer graphs.

VIII. Conclusion

In this paper, we propose a novel $\mathbf{k}$-dense subgraph model for multilayer graphs, which enforces explicit layer-wise density constraints. Building upon this model, we study the problem of multilayer density decomposition. For a single $D_{\mathbf{k}}$ computation, we design the LCVSP framework, which combines network flow techniques, core-based pruning, and layer-constrained propagation for fast convergence. For full decomposition, we present the HPDD and HPDD+ algorithms, leveraging the *density lattice* structure to enable scalable and memory-efficient exploration of the solution space. Extensive experiments demonstrate that our model consistently discovers higher-quality subgraphs compared to other models, while our algorithms achieve superior efficiency and scalability.

IX. AI-Generated Content Acknowledgement

Portions of this manuscript, specifically minor English editing and language polishing, were assisted by ChatGPT (OpenAI). All conceptual ideas, technical formulations, algorithm designs, theoretical analyses, and experimental results were entirely conceived, developed, and validated by the authors.

References

[1] M. E. Dickison, M. Magnani, and L. Rossi, *Multilayer social networks*. Cambridge University Press, 2016.

[2] S. Chen, H. Qian, Y. Wu, C. Chen, and X. Wang, "Efficient adoption maximization in multi-layer social networks," in *2019 International Conference on Data Mining Workshops (ICDMW)*. IEEE, 2019, pp. 56–60.

[3] B. Zhao, S. Hu, X. Li, F. Zhang, Q. Tian, and W. Ni, "An efficient method for protein function annotation based on multilayer protein networks," *Human genomics*, vol. 10, pp. 1–15, 2016.

[4] X. Liu, E. Maiorino, A. Halu, K. Glass, R. B. Prasad, J. Loscalzo, J. Gao, and A. Sharma, "Robustness and lethality in multilayer biological molecular networks," *Nature communications*, vol. 11, no. 1, p. 6043, 2020.

[5] B. Han, Y. Wei, Q. Wang, F. M. D. Collibus, and C. J. Tessone, "Mt 2 ad: multi-layer temporal transaction anomaly detection in ethereum networks with gnn," *Complex & Intelligent Systems*, vol. 10, no. 1, pp. 613–626, 2024.

[6] X. Huang, D. Chen, T. Ren, and D. Wang, "A survey of community detection methods in multilayer networks," *Data Mining and Knowledge Discovery*, vol. 35, pp. 1–45, 2021.

[7] X. Du, R. Jin, L. Ding, V. E. Lee, and J. H. Thornton Jr, "Migration motif: a spatial-temporal pattern mining approach for financial markets," in *Proceedings of the 15th ACM SIGKDD international conference on knowledge discovery and data mining*, 2009, pp. 1135–1144.

[8] A. F. Colladon and E. Remondi, "Using social network analysis to prevent money laundering," *Expert Systems with Applications*, vol. 67, pp. 49–58, 2017.

[9] H. Zhang, C. Zhuge, and X. Yu, "Identifying hub stations and important lines of bus networks: A case study in xiamen, china," *Physica A: Statistical Mechanics and Its Applications*, vol. 502, pp. 394–402, 2018.

[10] E. Galimberti, F. Bonchi, and F. Gullo, "Core decomposition and densest subgraph in multilayer networks," in *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*, 2017, pp. 1807–1816.

[11] B. Liu, F. Zhang, C. Zhang, W. Zhang, and X. Lin, "Corecube: Core decomposition in multilayer graphs," in *Web Information Systems Engineering–WISE 2019: 20th International Conference, Hong Kong, China, January 19–22, 2020, Proceedings 20*. Springer, 2019, pp. 694–710.

[12] F. Hashemi, A. Behrouz, and L. V. Lakshmanan, "Firmcore decomposition of multilayer networks," in *Proceedings of the ACM Web Conference 2022*, 2022, pp. 1589–1600.

[13] R.-A. Wang, D. Liu, and Z. Zou, "Focuscore decomposition of multilayer graphs," in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2024, pp. 2792–2804.

[14] A. Behrouz, F. Hashemi, and L. V. Lakshmanan, "Firmtruss community search in multilayer networks," *arXiv preprint arXiv:2205.00742*, 2022.

[15] Y. Zhang, R.-H. Li, Q. Zhang, H. Qin, and G. Wang, "Efficient algorithms for density decomposition on large static and dynamic graphs," *Proceedings of the VLDB Endowment*, vol. 17, no. 11, pp. 2933–2945, 2024.

[16] V. Batagelj and M. Zaversnik, "An o (m) algorithm for cores decomposition of networks," *arXiv preprint cs/0310049*, 2003.

[17] A. V. Goldberg, "Finding a maximum density subgraph," 1984.

[18] G. Borradaile, T. Migler, and G. Wilfong, "Density decompositions of networks," in *Complex Networks IX: Proceedings of the 9th Conference on Complex Networks CompleNet 2018 9*. Springer, 2018, pp. 15–26.

[19] Y. Sun, J. Han, X. Yan, P. S. Yu, and T. Wu, "Pathsim: Meta path-based top-k similarity search in heterogeneous information networks," *Proceedings of the VLDB Endowment*, vol. 4, no. 11, pp. 992–1003, 2011.

[20] I. Bezáková, "Compact representations of graphs and adjacency testing," 2000.

[21] M. Blumenstock, "Fast algorithms for pseudoarboricity," in *Proceedings of the Eighteenth Workshop on Algorithm Engineering and Experiments, ALENEX 2016, Arlington, Virginia, USA, January 10, 2016*, M. T. Goodrich and M. Mitzenmacher, Eds. SIAM, 2016, pp. 113–126.

[22] E. Galimberti, F. Bonchi, F. Gullo, and T. Lanciano, "Core decomposition in multilayer networks: Theory, algorithms, and applications," *ACM Transactions on Knowledge Discovery from Data (TKDD)*, vol. 14, no. 1, pp. 1–40, 2020.

[23] R. Agrawal, R. Srikant *et al.*, "Fast algorithms for mining association rules," in *Proc. 20th Int. Conf. Very Large Data Bases, VLDB*, vol. 1215, 1994, pp. 487–499.

[24] L. Chang and L. Qin, *Cohesive subgraph computation over large sparse graphs: algorithms, data structures, and programming techniques*. Springer, 2018.

[25] M. E. Dickison, M. Magnani, and L. Rossi, *Multilayer Social Networks*. Cambridge University Press, 2016.

[26] E. Omodei, M. De Domenico, and A. Arenas, "Characterizing interactions in online social networks during exceptional events," *Frontiers in Physics*, vol. 3, p. 59, 2015.

[27] J. A. Baggio, S. B. BurnSilver, A. Arenas, J. S. Magdanz, G. P. Kofinas, and M. De Domenico, "Multiplex social ecological network analysis reveals how social changes affect community robustness more than resource depletion," *Proceedings of the National Academy of Sciences*, vol. 113, no. 48, pp. 13 708–13 713, 2016.

[28] A. Mislove, H. S. Koppula, K. P. Gummadi, P. Druschel, and B. Bhattacharjee, "Growth of the flickr social network," in *Proceedings of the first workshop on Online social networks*, 2008, pp. 25–30.

[29] R. Zhu, Z. Zou, and J. Li, "Diversified coherent core search on multilayer graphs," in *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 2018, pp. 701–712.

[30] R. D. Luce and A. D. Perry, "A method of matrix analysis of group structure," *Psychometrika*, vol. 14, no. 2, pp. 95–116, 1949.

[31] M. A. Reed, C. Zhou, C. Muller, T. Burgin, and J. Tour, "Conductance of a molecular junction," *Science*, vol. 278, no. 5336, pp. 252–254, 1997.

[32] "BioGRID: Biological general repository for interaction datasets," https://thebiogrid.org, 2025, accessed: 2025-01-02.

[33] "Gene Ontology consortium: Data downloads," http://current.geneontology.org/products/pages/downloads.html, 2025, accessed: 2025-01-02.

[34] E. Altman, J. Blanuša, L. von Niederhäusern, B. Egressy, A. Anghel, and K. Atasu, "Realistic synthetic financial transactions for anti-money laundering models," in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 29 851–29 874. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/5f38404edff6f3f642d6fa5892479c42-Paper-Datasets_and_Benchmarks.pdf

[35] R. Jensen, J. Ferwerda, K. Jorgensen, E. Jensen, M. Borg, M. Krogh, J. Jensen, and A. Iosifidis, "A synthetic data set to benchmark anti-money laundering methods," *Scientific data*, vol. 10, no. 1, pp. 1–10, Dec. 2023, publisher Copyright: © 2023, Springer Nature Limited.

[36] V. Jethava and N. Beerenwinkel, "Finding dense subgraphs in relational graphs," in *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 2015, pp. 641–654.

[37] N. Azimi-Tafreshi, J. Gómez-Gardenes, and S. Dorogovtsev, "k-core percolation on multiplex networks," *Physical Review E*, vol. 90, no. 3, p. 032816, 2014.

[38] H. Huang, Q. Linghu, F. Zhang, D. Ouyang, and S. Yang, "Truss decomposition on multilayer graphs," in *2021 IEEE International Conference on Big Data (Big Data)*. IEEE, 2021, pp. 5912–5915.

[39] N. Tatti and A. Gionis, "Density-friendly graph decomposition," in *Proceedings of the 24th International Conference on World Wide Web*, 2015, pp. 1089–1099.

[40] M. Danisch, T.-H. H. Chan, and M. Sozio, "Large scale density-friendly graph decomposition via convex programming," in *Proceedings of the 26th International Conference on World Wide Web*, 2017, pp. 233–242.

[41] L. Qin, R.-H. Li, L. Chang, and C. Zhang, "Locally densest subgraph discovery," in *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2015, pp. 965–974.

[42] Y. Zhang, R.-H. Li, Q. Zhang, H. Qin, L. Qin, and G. Wang, "Density decomposition of bipartite graphs," *Proceedings of the ACM on Management of Data*, vol. 3, no. 1, pp. 1–25, 2025.